

SMART HOME IoT SECURITY SYSTEM

Implementation Research Report

*Security-Focused Local IoT Architecture with Mutual TLS, VLAN
Segmentation, and Real-Time Anomaly Detection*

Project: Smart Home IoT Automation & Security Platform

Stack: FastAPI · Next.js 15 · PostgreSQL/TimescaleDB · Mosquitto MQTT · Docker **Date:**
June 2026

Report Type: Research Implementation Report

Prepared by: Asadul Islam Arif

Table of Contents

1	Executive Summary
2	Research Context & Objectives
3	System Architecture Overview
3.1	High-Level Architecture
3.2	Service Components
3.3	Network Segmentation (VLAN Design)
3.4	Data Flow
4	Security Implementation
4.1	Transport Layer Security - TLS 1.3
4.2	Mutual TLS (mTLS) for MQTT Devices
4.3	Per-Device Certificate Management
4.4	Zero-Touch Device Provisioning
4.5	Authentication & JWT Session Management
4.6	Role-Based Access Control (RBAC)
4.7	MQTT Access Control List (ACL)
4.8	IP Blocking & Rate Limiting
4.9	Security Audit Trail & Event Management
4.10	Anomaly Detection (Z-Score & Isolation Forest)
4.11	Admin Security Actions
5	Backend Implementation (FastAPI)
5.1	API Design & Routing
5.2	Database Layer (PostgreSQL + TimescaleDB)
5.3	MQTT Bridge Service
5.4	Telegram Notification Bot
5.5	Structured Logging
6	Frontend Implementation (Next.js 15)
6.1	Dashboard & Device Management
6.2	Security Page
6.3	Room & Favorites Management
6.4	Real-Time Updates via WebSocket
6.5	Responsive Design & Mobile Support
7	Deployment & Infrastructure
7.1	Docker Compose Architecture
7.2	Nginx Reverse Proxy
7.3	Hardware Deployment (Office PC / Mini PC)
7.4	Backup & Data Retention
8	Voice Control Integration
9	Maintainability & Troubleshooting
9.1	Observability & Logging
9.2	Diagnostic Workflows
9.3	Certificate Renewal Workflow
10	User Interface & User Experience
11	Testing & Validation
12	Limitations & Future Work
13	Conclusion

1. Executive Summary

This report documents the design, implementation, and evaluation of a security-first Smart Home IoT Automation Platform. The system addresses a critical gap identified in consumer IoT deployments: the systematic absence of transport encryption, device authentication, and real-time threat visibility in off-the-shelf smart home solutions.

The platform was built as a fully local, cloud-independent system deployable on commodity hardware (a used office PC or mini PC). It is designed to be local-first: every core function (device control, telemetry storage, anomaly detection, the security dashboard, and the offline voice control subsystem) runs on the local network and continues to operate completely without internet access. It enforces TLS 1.3 on every communication channel, requires mutual TLS (mTLS) client certificates for each IoT device, segments devices across dedicated VLANs, and surfaces security events to administrators in real time through a purpose-built web dashboard. An optional Telegram integration can be enabled to provide out-of-band push notifications for anomalies and critical security events, and an out-of-band channel for password reset; the system is fully operational without it.

Key outcomes of the implementation:

- End-to-end encryption enforced: TLS 1.3 on all HTTP, WebSocket, and MQTT traffic.
- Per-device identity: Each IoT device holds a unique X.509 certificate signed by a local CA.
- Network isolation: 6 VLANs prevent lateral movement between device classes, with security cameras placed on a dedicated camera VLAN separate from general IoT.
- Real-time threat detection: Z-score and Isolation Forest anomaly detection on telemetry streams.
- Complete audit trail: Every authentication attempt, certificate event, and admin action logged.
- One-click remediation: Admins can block IPs, renew certificates, force-logout users, or deactivate accounts directly from the UI.
- Local-first, no cloud dependency: every core function runs on the local network; the system continues to operate completely without internet access. All data remains on-premises with no vendor lock-in. Telegram is supported as an optional out-of-band alerting channel only.
- Voice control: English and Bengali wake-word recognition via Rhasspy/Vosk.
- Hardware flexibility: runs on any used office PC or mini PC (NUC, Beelink, etc.); no proprietary hardware required.

Research Relevance: This implementation directly validates the security architecture proposed in the associated research paper, demonstrating that strong IoT security posture is achievable at consumer price points without sacrificing usability.

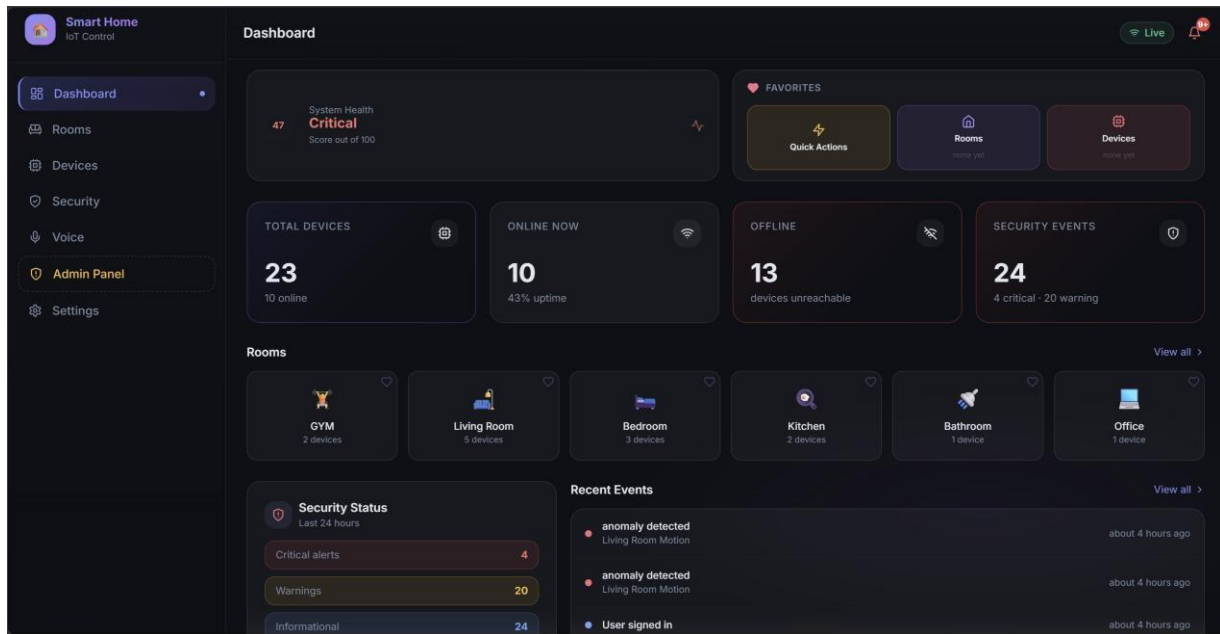


Figure 1 – Dashboard: showing devices, rooms, favorites and security overview

2. Research Context & Objectives

2.1 Problem Statement

The proliferation of consumer IoT devices has introduced significant security risks into residential and small-business environments. Studies consistently find that the majority of commercially available smart home devices:

- Transmit telemetry over plaintext protocols (MQTT port 1883, HTTP port 80).
- Rely on shared or default credentials for device authentication.
- Route all traffic through vendor cloud infrastructure, creating privacy and availability risks.
- Lack per-device revocable identities, making compromise containment difficult.
- Provide no real-time visibility into anomalous behaviour.

2.2 Research Objectives

- RO1: Design and implement a local-first smart home platform that operates completely without internet connectivity, with no mandatory cloud dependencies. An optional Telegram integration may be enabled for out-of-band push alerts and password reset, and is the only feature in the platform that uses external connectivity.
- RO2: Enforce mutual TLS (mTLS) authentication for every IoT device using per-device X.509 certificates.
- RO3: Demonstrate network micro-segmentation through VLAN isolation of device classes.
- RO4: Implement real-time anomaly detection on sensor telemetry streams using statistical and machine-learning methods.
- RO5: Build an operator-facing security dashboard that makes the security posture visible and actionable.
- RO6: Validate that strong security controls can be implemented on commodity hardware (USD 120 in Bangladesh, USD 224 in Australia) without degrading user experience.

2.3 Scope

The implementation covers: device registration and lifecycle management, encrypted MQTT communication, web and mobile-responsive administration interface, time-series telemetry storage and

analytics, security event logging and acknowledgement workflows, IP blocking, role-based access control, voice command integration, and containerised deployment. Out of scope: physical hardware penetration testing, formal threat modelling (Stride/DREAD), and external PKI integration.

3. System Architecture Overview

3.1 High-Level Architecture

The system follows a layered microservices architecture. All external access passes through an Nginx reverse proxy that handles TLS termination. Behind it, four core services operate on an isolated Docker network:

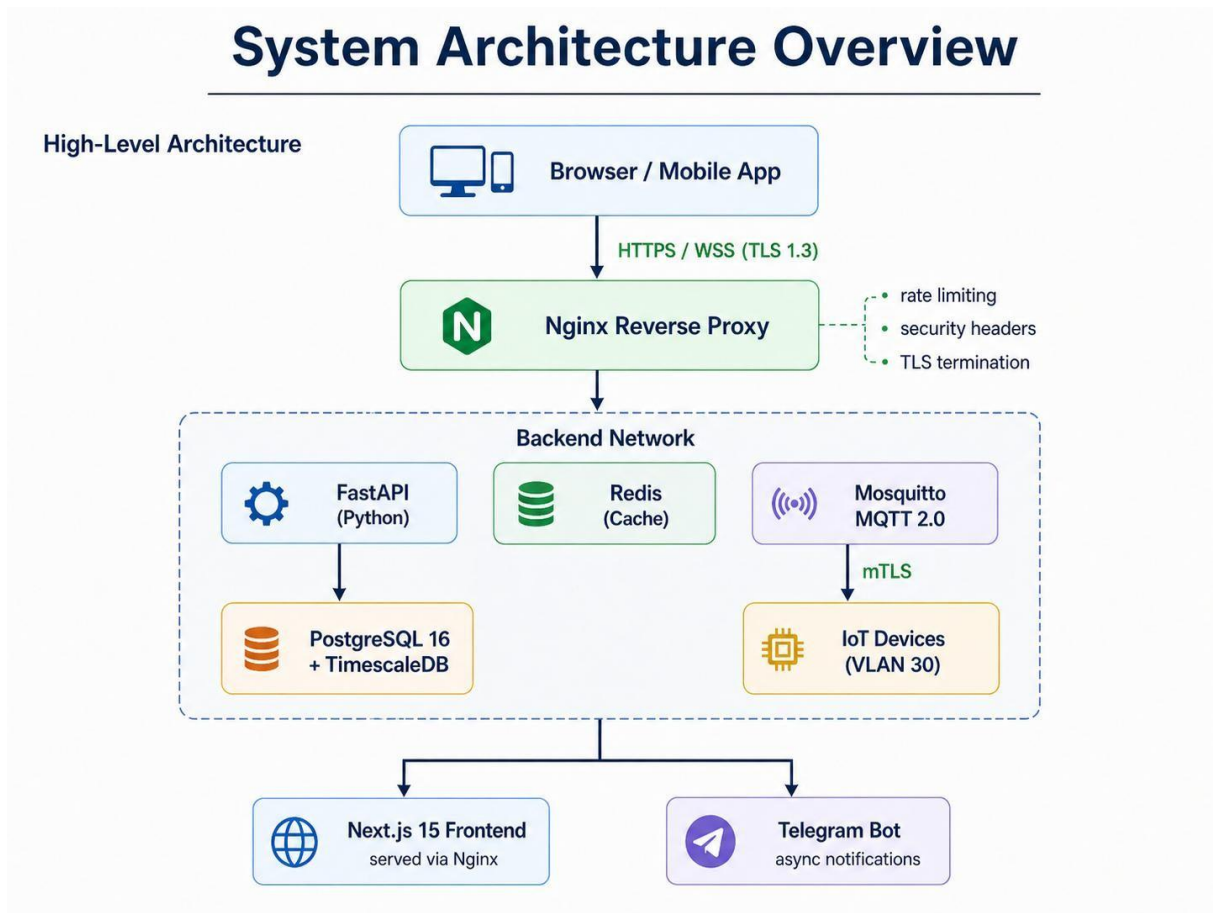


Figure 2 – System Architecture Overview

☐	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Actions
☐	infrastructure	-	-	-	2.64%	421.17MB / 91.8G	☐ ☐ ☐
☐	web-1	11bed7d2a0eb	infrastructure	3001:3000	0%	34.37MB / 15.3Gi	☐ ☐ ☐
☐	nginx-1	56b91a58ff2d	nginx:alpin	443:443	0%	17.41MB / 15.3Gi	☐ ☐ ☐
☐	api-1	bfd4cce56179	infrastructure	8000:8000	1.18%	295.4MB / 15.3Gi	☐ ☐ ☐
☐	mosquitto-1	479d0732b566	infrastructure	1883:1883	0.68%	22.73MB / 15.3Gi	☐ ☐ ☐
☐	postgres-1	bf631fa0dd60	timescale/t	5432:5432	0.02%	44.93MB / 15.3Gi	☐ ☐ ☐
☐	redis-1	a0e88e574faa	redis:7-alpi	6379:6379	0.76%	6.33MB / 15.3GB	☐ ☐ ☐

Figure 3 – Docker Containers up and running

3.2 Service Components

Service	Technology	Port	Role
Nginx	nginx:alpine	80 / 443	TLS termination, rate limiting, reverse proxy
API	FastAPI + Uvicorn	8000	REST API, WebSocket hub, MQTT bridge
Web UI	Next.js 15 (Node 20)	3000	React admin dashboard
Database	PostgreSQL 16 + TimescaleDB	5432	Relational + time-series storage
Cache	Redis 7	6379	Session cache, rate-limit counters
MQTT Broker	Mosquitto 2.0	1884/8883/9001	TLS + mTLS broker for IoT devices (8883), plus an internal authenticated plaintext listener (1884) for bootstrap and the voice service
Voice	Rhasspy + Vosk	12101	Offline wake-word + speech-to-text
Simulator	Python MQTT publisher	-	Development: 10 virtual IoT devices
Telegram Bot	python-telegram-bot 20	-	Optional out-of-band push alerts (anomalies, security events)

3.3 Network Segmentation - VLAN Design

A six-VLAN topology isolates device classes from one another. Security cameras are placed on a dedicated VLAN of their own so that, even within the IoT estate, a compromised lightbulb cannot reach the camera stream and vice versa. Even if a smart device is compromised, it cannot directly reach the management or user VLANs. This is a central tenet of the research paper's security model.

VLAN ID	Name	Subnet	Devices	Access Policy
99	Management	10.10.99.0/24	pfSense, Technitium DNS, managed switch, AP, admin laptop	Admin-only; isolated from user and IoT
20	NAS / Storage	10.10.20.0/24	NAS, file/media storage, backups	Wired access port + wireless SSID mapped to VLAN 20
30	IoT network	10.10.30.0/24	Lights, locks, sensors, plugs	Outbound MQTT (8883) and internet; blocked from RFC1918 private networks
40	General user	10.10.40.0/24	Laptops, phones, end-user dashboard clients	Web dashboard (HTTPS) only; blocked from MANAGEMENT_HOSTS
50	Guest	10.10.50.0/24	Guest phones, visitor devices	Internet only; no LAN
60	Cameras	10.10.60.0/24	IP cameras (indoor / outdoor), NVR appliance	RTSP / ONVIF inbound only; isolated from IoT VLAN 30

Table 1 - VLAN segmentation policy

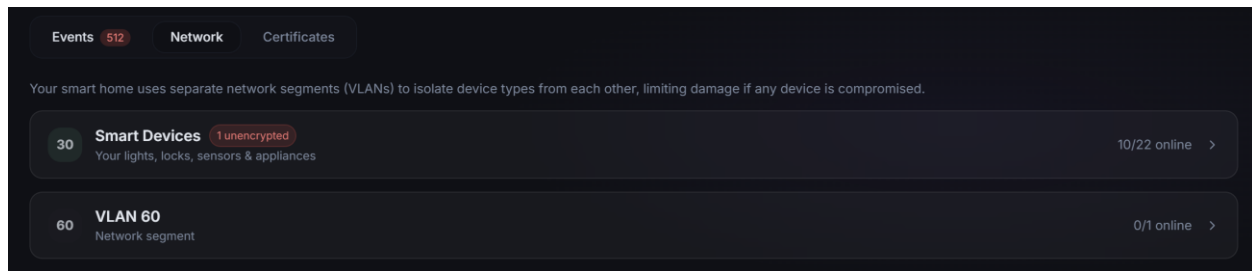


Figure 4 - Security page: Network tab showing VLAN segmentation with five isolated zones

3.4 Data Flow

IoT Device → MQTT (TLS 8883) → Mosquitto Broker → FastAPI MQTT Bridge → PostgreSQL (telemetry) + Redis (state) + WebSocket (real-time push to browser). Commands flow in reverse: Web UI → FastAPI → MQTT publish → Device.

- Device powers on → presents mTLS client certificate to Mosquitto → broker validates against local CA.
- Device publishes telemetry on topic `smarthome/{device_id}/telemetry` (QoS 1).
- MQTT Bridge receives message, stores to `device_telemetry` (TimescaleDB hypertable), runs Z-score check.
- If anomaly detected: log `security_event` (critical/warning), push WebSocket notification to all connected admins, send Telegram alert.
- Admin views dashboard → clicks Acknowledge → event marked reviewed, moves to 30-day archive.
- Admin sends command (e.g., `turn_on`) → FastAPI publishes to `smarthome/{device_id}/command` → device actuates.

4. Security Implementation

This section is the core of the research paper implementation. Each sub-section maps directly to a security control described in the paper.

4.1 Transport Layer Security - TLS 1.3

All communication channels enforce TLS 1.3 as the minimum protocol version. Older protocol versions (TLS 1.0, 1.1, 1.2) are explicitly rejected at both the Nginx reverse proxy and the Mosquitto MQTT broker.

Nginx Configuration

```
ssl_protocols TLSv1.3; ssl_prefer_server_ciphers on; ssl_ciphers
TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256; add_header
X-Frame-Options DENY; add_header X-Content-Type-Options nosniff;
add_header X-XSS-Protection "1; mode=block";
```

Mosquitto MQTT Broker

```
listener 8883 tls_version tlsv1.3 cafile
/mosquitto/certs/ca/ca.crt certfile
/mosquitto/certs/server/server.crt keyfile
/mosquitto/certs/server/server.key
require_certificate true
```

The Certificate Authority is a locally-generated 4096-bit RSA CA with a 10-year validity period, created during initial setup via `scripts/setup/generate-certs.sh`. The CA private key never leaves the deployment host.

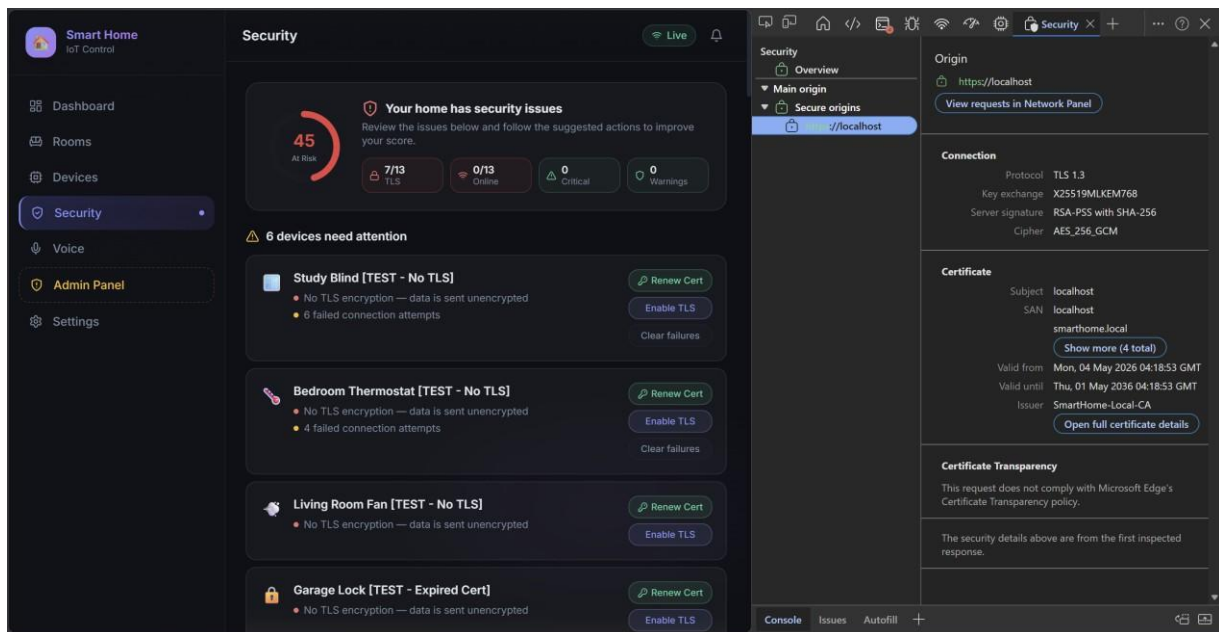


Figure 5 - Browser certificate inspector confirming TLS 1.3 is in use

4.2 Mutual TLS (mTLS) for MQTT Devices

Standard TLS only authenticates the server to the client. The system goes further by requiring every IoT device to present a valid client certificate during the MQTT TLS handshake - this is mutual TLS (mTLS). A device without a valid certificate signed by the local CA is refused connection regardless of credentials.

This eliminates an entire class of attacks: a rogue device or network intruder cannot publish fabricated telemetry or receive commands even if they know the MQTT credentials, because they do not possess a CA-signed client certificate.

```
# FastAPI MQTT Bridge - mTLS context setup
tls_context =
ssl.create_default_context(ssl.Purpose.SERVER_AUTH)
tls_context.load_verify_locations(settings.MQTT_TLS_
CA_CERT) tls_context.load_cert_chain(
settings.MQTT_TLS_CLIENT_CERT,
settings.MQTT_TLS_CLIENT_KEY
) client.tls_set_context(tls_context)
```

Code 1 - mTLS context initialisation in the MQTT Bridge service

Security Impact: mTLS reduces the MQTT attack surface to zero for unauthenticated devices. All 10 device types (lights, locks, thermostats, sensors, cameras, plugs, fans, blinds) are required to present certificates.

4.3 Per-Device Certificate Management

Each IoT device is issued a unique X.509 certificate during provisioning. Validity is chosen based on whether the device supports over-the-air (OTA) cert updates: OTA-capable devices receive a 1-year certificate that the platform renews automatically over MQTT before expiry, while non-OTA devices (e.g. fixed-firmware Zigbee sensors or off-the-shelf Sonoff units) receive a 10-year certificate so that field intervention is rare. Certificates can also be renewed manually at any time via the Security page in the web UI.

Certificate Generation Parameters

Parameter	Value
Key Algorithm	RSA 2048-bit
Signature Hash	SHA-256 with RSA
Validity Period	1 year (OTA devices, auto-renewed) / 10 years (non-OTA)
Subject (CN)	device_{device_id} (unique per device)
CA Key Size	RSA 4096-bit
CA Validity	3650 days (10 years)
BasicConstraints	CA:FALSE (device certs cannot sign other certs)
Renewal API	POST /api/security/devices/{id}/renew-certificate
Renewal Output	device.crt + device.key + ca.crt (PEM, shown once)

Certificate Renewal Workflow

- Admin navigates to Security → Certificates tab.
- Devices with certificates expiring within 30 days are highlighted in amber; expired in red.
- Admin clicks 'Renew' → API generates new 2048-bit RSA keypair + certificate.
- A modal displays the three PEM files (certificate, private key, CA cert) with copy/download buttons.
- Private key is shown only once - admin must flash the files to the device immediately.
- Database updated: tls_enabled=true, certificate_expiry set to +1 year for OTA devices or +10 years for non-OTA devices.
- Security event logged: certificate_renewed (severity: info).

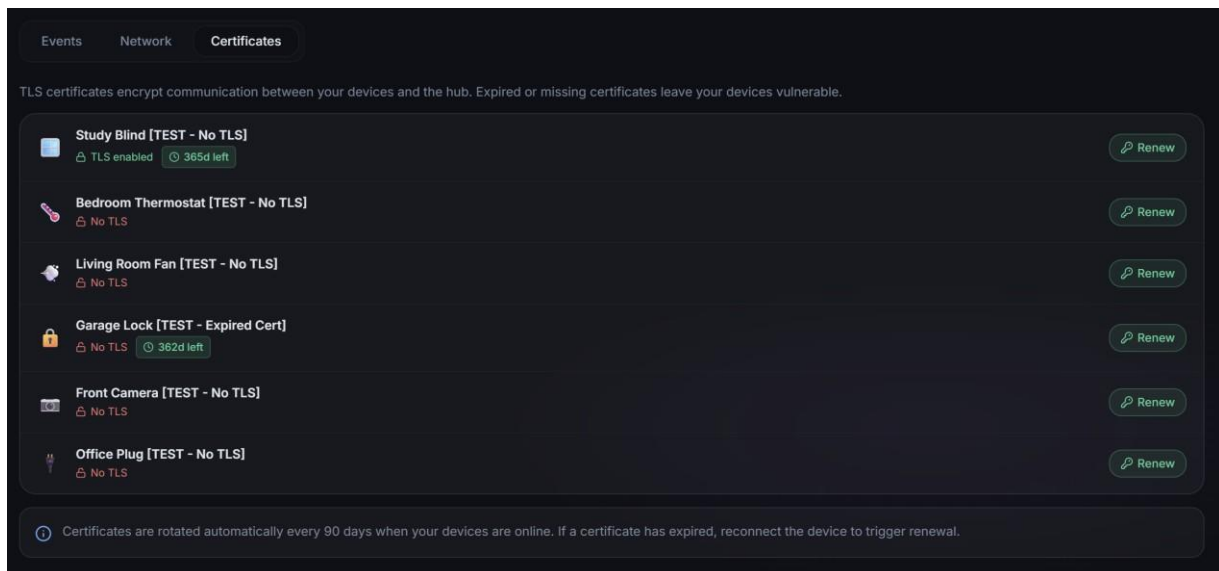


Figure 6 - Certificate management tab: expiry status shown as green, amber, and red indicators

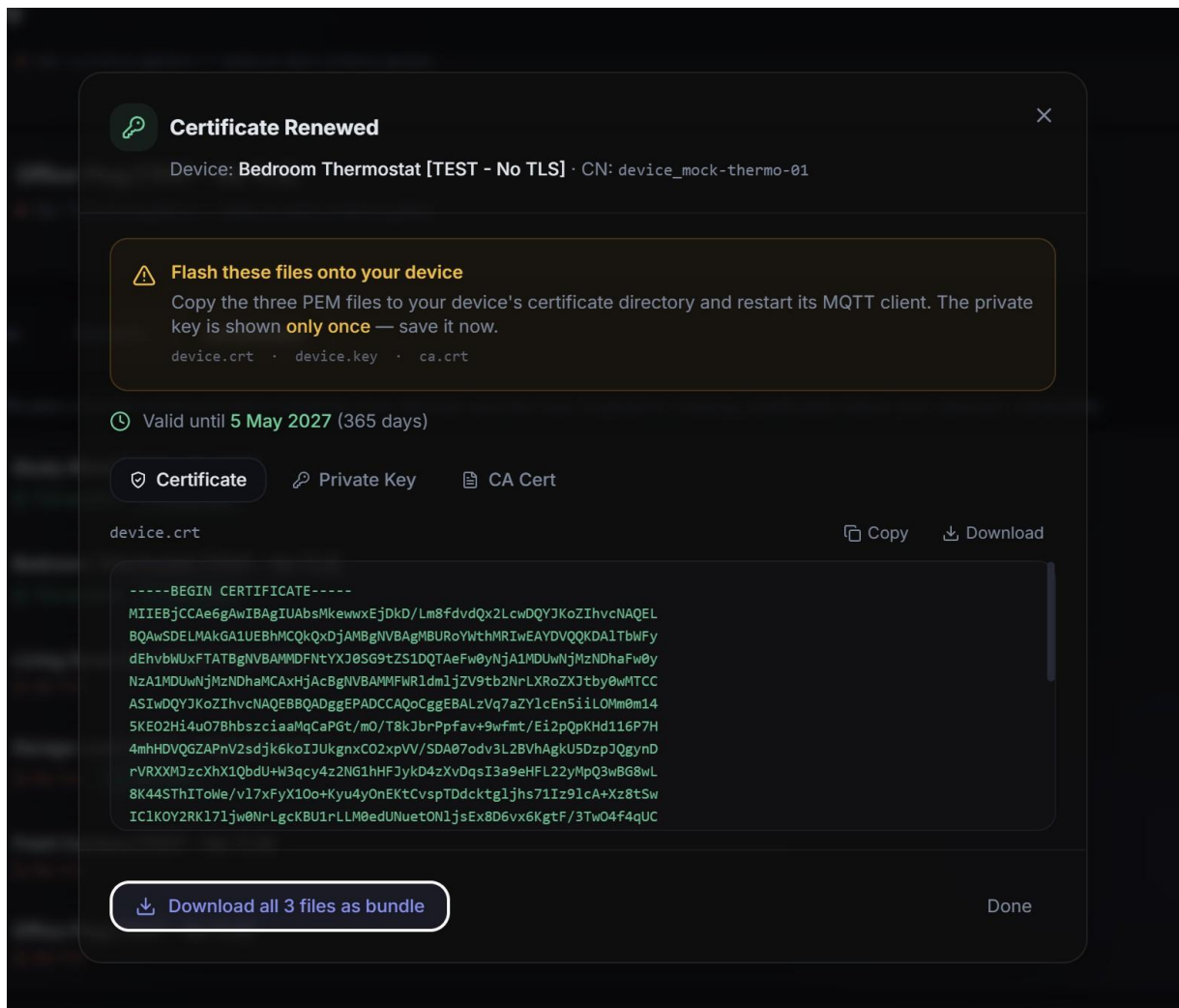


Figure 7 - Certificate Renewal Modal: three PEM files with copy and download buttons

4.4 Zero-Touch Device Provisioning

Bringing a new IoT device onto a secure mTLS network is normally a manual, error-prone procedure: generate a cert and key on the server, copy three PEM files plus an MQTT username and password onto the device by flash or SSH, and then add the username to the broker's password file by hand. Every additional device adds another round of touching it. The platform addresses this with a zero-touch provisioning flow that hands a new device a unique identity automatically the first time it powers on, leaving exactly one credential — the shared bootstrap password — that must be baked into the firmware before shipping. The benefit is twofold: onboarding a tenth or hundredth device costs the same as onboarding the first, and each device still ends up with its own per-device certificate rather than a shared key, so the mTLS guarantee is preserved at scale.

The flow runs entirely over MQTT. A new device connects to the broker's internal port 1884 using the bootstrap credential and publishes its identity — device type, MAC address, firmware version, and whether it supports over-the-air certificate updates — to provision/{device_id}/register. The API picks up the message, upserts the device into the database in a pending state, generates an individual MQTT username, a random 48-character password, and a fresh X.509 client certificate signed by the local CA (1-year validity for OTA-capable devices, 10-year for non-OTA), adds the new username to Mosquitto's

password file via the broker container's Docker socket, and sends a SIGHUP so the change is picked up without a broker restart. The full credential bundle is then published back to `provision/{device_id}/credentials`. The device persists the bundle to its NVS flash, disconnects from the bootstrap listener, and reconnects on port 8883 under its own identity with mTLS. From that point on it is indistinguishable from a hand-provisioned device.

For visibility and override, the API exposes a small admin surface under `/api/provision/`. `GET /pending` lists devices that have announced themselves but have not yet been approved, `GET /devices` lists provisioned devices and flags any whose certificate is due to expire within 30 days, `POST /{device_id}/approve` manually triggers credential issuance, `POST /{device_id}/revoke` removes the device's Mosquitto user and marks it revoked in the database, and `POST /{device_id}/renew-cert` force-renews a single device's certificate on demand. All five endpoints are admin-only and produce audit events on every action, so the zero-touch path remains observable from the Security page rather than becoming an invisible side channel.

A background async task in the API runs on a 24-hour cycle and queries for OTA-capable devices whose certificate expires within the next 30 days. For each match it issues a new certificate (keeping the existing MQTT password unchanged), updates the database, and publishes the renewed certificate over MQTT on `smarthome/{device_id}/cert_update`. Devices subscribe to that topic at startup and, on receipt, persist the new cert to flash and reboot to apply it. The practical effect is that, for the working life of an OTA-capable device, certificate rotation is invisible to the operator — no field visits, no flashing tools, and no expired-cert outages. Non-OTA devices opt out of this path by virtue of their 10-year cert and the matching `docs/non-ota-device-guide.md` walk-through for the rare manual renewal.

To support the new workflow, the devices table was extended with four columns — `ota_enabled` (boolean), `provision_state` (enum: `unprovisioned` | `pending` | `provisioned` | `revoked`, constraint-enforced), `cert_serial` (the X.509 serial number for tracking), and `mqtt_password_hash` (kept for audit and reference) — together with an index on the new state column and a partial index on `certificate_expiry` filtered to provisioned OTA devices so the daily renewal query stays cheap. The accompanying data migration auto-marks devices that already had an MQTT username as provisioned, so existing deployments inherit the new workflow without re-onboarding. A reference ESP32 Arduino sketch ships alongside the platform to make the protocol trivial to validate on real hardware: it derives a stable device ID from the lower three bytes of the MAC, persists credentials to NVS flash, and contains the two state machines the protocol needs — `connectBootstrap` (plaintext on 1884, publish register, wait for credentials, save them) and `connectNormal` (mTLS on 8883, subscribe to command and `cert_update` topics, publish telemetry every 30 s). The sketch is intended as a starting point for vendor firmware rather than a finished product, but it documents the full protocol on the device side and works end-to-end against the deployed broker.

4.5 Authentication & JWT Session Management

Human users authenticate via email/password. Passwords are hashed using bcrypt (via passlib) before storage - plaintext passwords are never stored or logged. A successful login returns a signed JSON Web Token (JWT) valid for 8 hours. The token lifetime was deliberately shortened from a longer working day so that a stolen token has at most one shift of usefulness before it must be re-issued, which materially reduces the impact of an undetected device or browser compromise.

JWT Token Specification

Field	Value	Purpose
Algorithm	HS256 (HMAC-SHA256)	Symmetric signature - server verifies own tokens
Secret	64-char random string	Configured in .env, never committed to repository
Expiry	8 hours (configurable)	Limits session hijacking window
Payload sub	User UUID	Maps token to database user record

Payload role	admin user	Embedded for fast RBAC without DB lookup
force_logout_at	Timestamp or NULL	Tokens issued before this timestamp are rejected

Force Logout Mechanism

If an admin suspects a user account has been compromised, they can execute a Force Logout from the Admin panel. This writes the current timestamp to `users.force_logout_at`. Every subsequent JWT validation checks that the token's `iat` (issued-at) is after `force_logout_at` - older tokens are immediately rejected, invalidating the session without requiring token revocation infrastructure.

```
# Token validation - force logout enforcement if
user['force_logout_at'] and \    payload['iat'] <
user['force_logout_at'].timestamp():    raise
HTTPException(401, 'Session has been revoked')
```

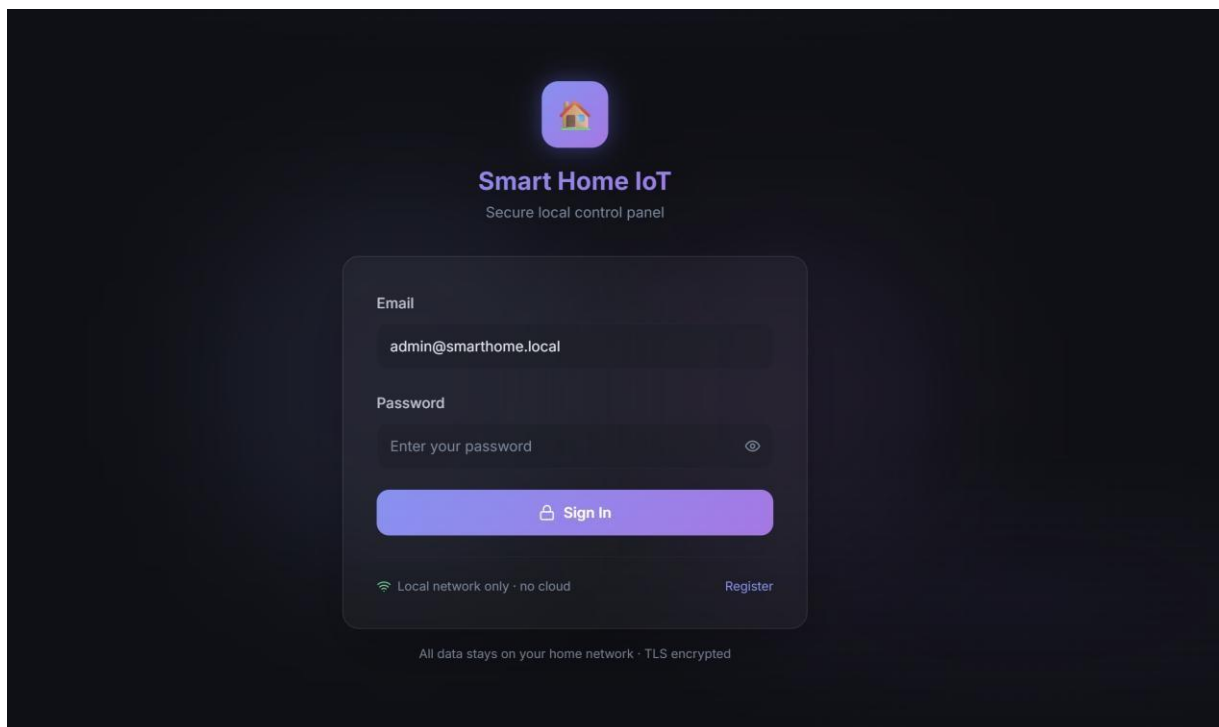


Figure 8 - Login page with email/password form and Forgot Password option

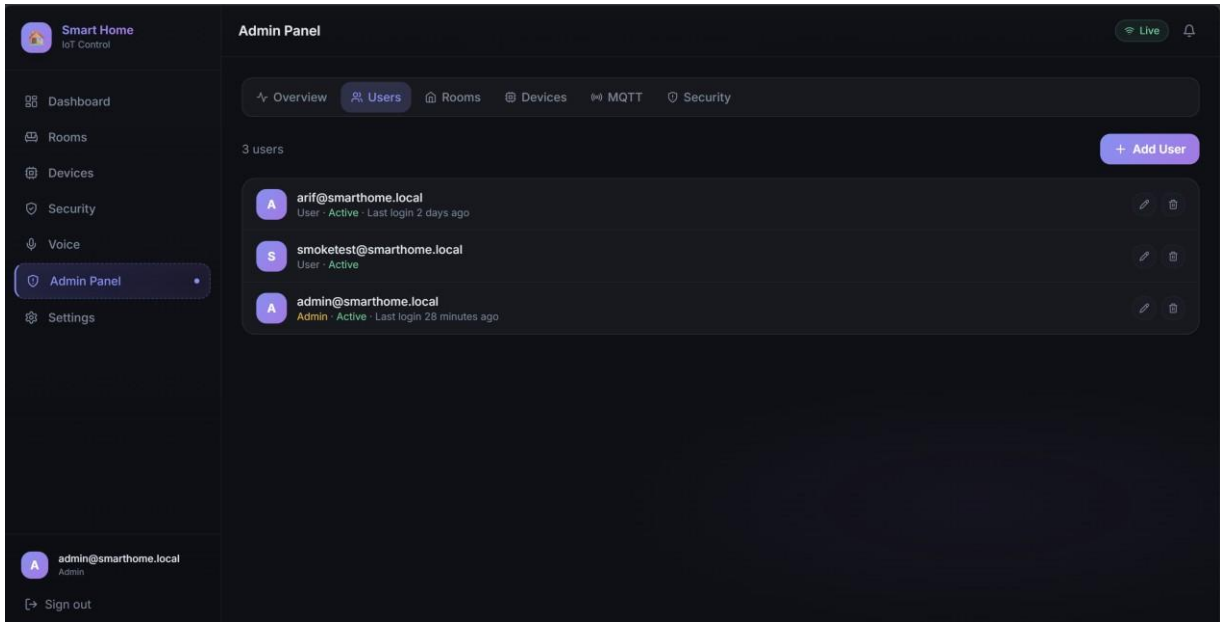


Figure 9 - Admin panel: user table with role badges, Force Logout, and Deactivate buttons

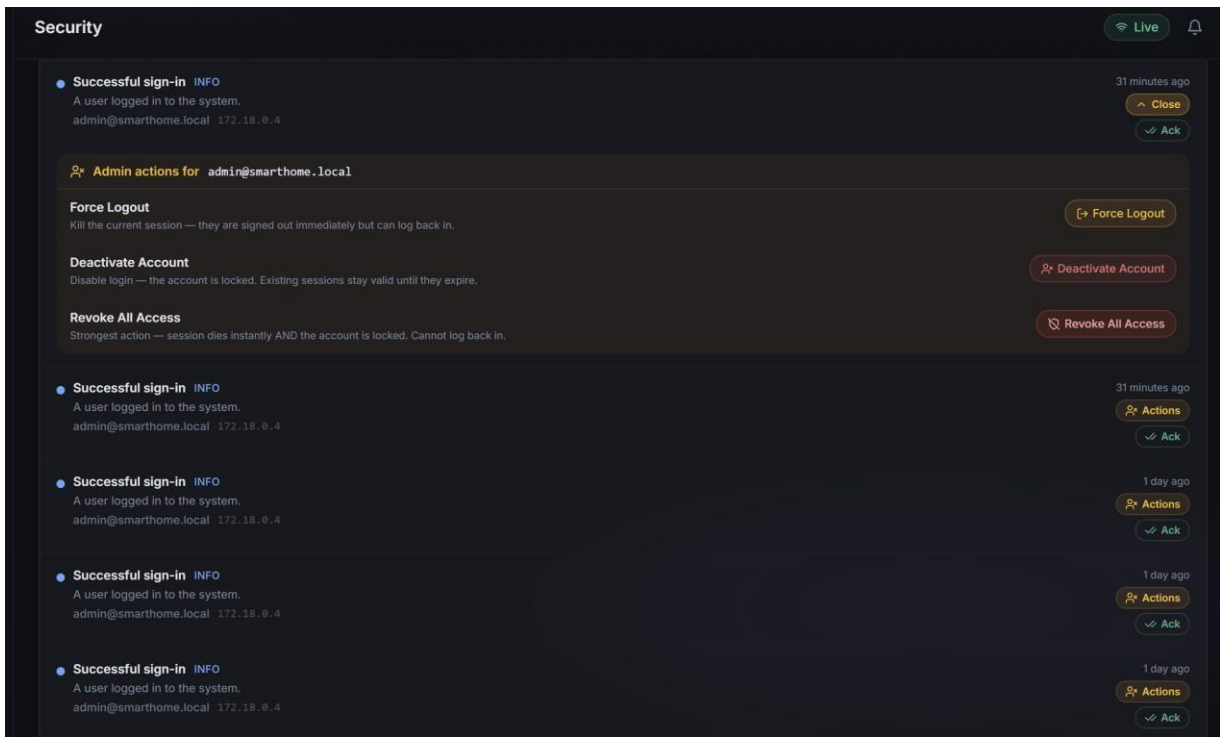


Figure 10 - Force logout confirmation before invalidating all active sessions

4.6 Role-Based Access Control (RBAC)

The system implements a two-tier RBAC model. Regular users can view devices and rooms and send commands to devices they have access to. Admins have unrestricted access including all security operations, user management, and device provisioning.

Capability	User	Admin
View dashboard & devices	Yes	Yes

Send device commands	Yes	Yes
Add / edit / delete devices	No	Yes
Create / manage rooms	No	Yes
View security events	Yes	Yes
Acknowledge security events	No	Yes
Block / unblock IP addresses	No	Yes
Renew device certificates	No	Yes
Force logout / deactivate users	No	Yes
Register new users	No	Yes
Bulk acknowledge events	No	Yes
Access Admin panel	No	Yes

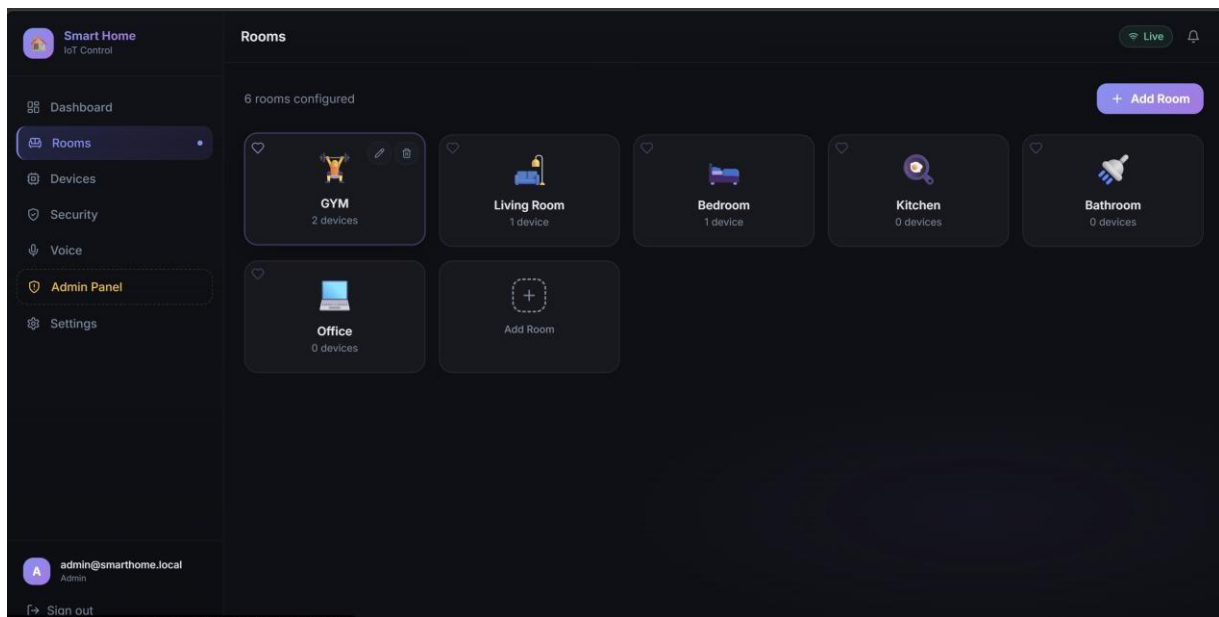


Figure 11 - Device card showing admin-only controls hidden from regular users

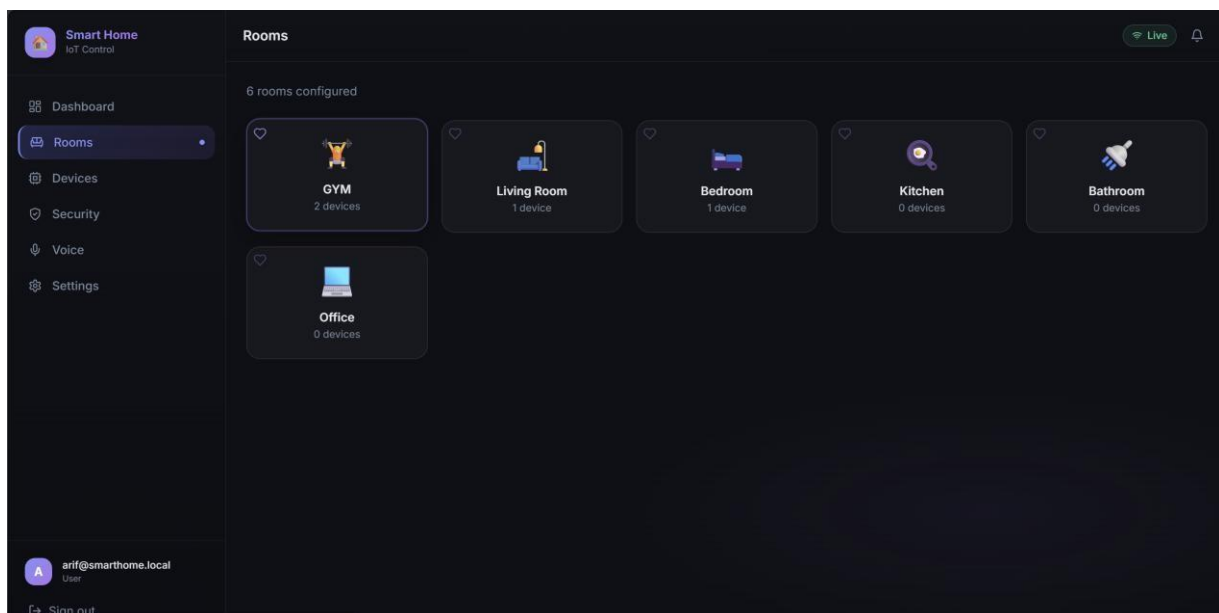


Figure 12 - RBAC in practice: regular user view without administrative options

4.7 MQTT Access Control List (ACL)

Mosquitto enforces topic-level ACLs. Each device's MQTT username is `device_{device_id}` and it may only publish telemetry/status on its own topic subtree, and only subscribe to its own command topic. It cannot read other devices' telemetry or publish commands.

```
# acl.conf - pattern substitutes %u with the MQTT username
user controller topic readwrite smarthome/#

# Per-device: can only talk on its own subtree
pattern write smarthome/%u/telemetry pattern
write smarthome/%u/status pattern read
smarthome/%u/command

user dashboard topic
read smarthome/#
```

Code 2 - MQTT ACL configuration (acl.conf)

Security Impact: A compromised device cannot read commands sent to other devices or inject false telemetry under another device's identity - topic isolation is enforced at the broker level, independent of application logic.

Beyond per-device patterns, the broker exposes two listeners with distinct trust profiles. The external listener on port 8883 enforces TLS 1.3, mTLS, and the ACL file together, so a connection that survives the handshake is still bound by topic-level permissions. An internal authenticated plaintext listener on port 1884 (only reachable from the Docker network, never exposed to a VLAN) carries traffic that cannot use mTLS yet: brand-new devices during their first-boot bootstrap, and the voice service which has no per-device identity to present. Each consumer has its own MQTT user with a tightly-scoped ACL: a write-only bootstrap user that can only publish to `provision/+` and receive credentials on its own subtree; a `rhasspy` user with read access to `smarthome/#` for observation and read/write on `rhasspy/#` for its own RPC; and the controller (the FastAPI bridge) extended with readwrite access to `provision/#` so it can deliver credential bundles and cert updates to devices. Devices additionally get read access to `smarthome/%u/cert_update`, which is the topic used for OTA certificate renewal. This layering means the platform can support zero-touch onboarding without weakening the mTLS guarantee that already protects steady-state traffic.

4.8 IP Blocking & Rate Limiting

Rate Limiting (Nginx)

The Nginx reverse proxy applies two separate rate-limit zones based on how sensitive the endpoint is. Auth endpoints (login, register, and password reset) are limited strictly to block automated attacks. The general API zone is more relaxed so normal dashboard use is never interrupted:


```
# 00-rate-limit.conf
# Auth endpoints - strict: 10 req/min per IP limit_req_zone
$binary_remote_addr zone=auth:10m rate=10r/m;
# General API - relaxed: 120 req/min per IP limit_req_zone
$binary_remote_addr zone=api:10m rate=120r/m;
# web.conf - auth routes use the strict zone (burst=3: a human can retry, a bot
cannot):
location ~ ^/api/auth/(login|public-register|reset-request|reset-confirm) {
    limit_req zone=auth burst=3 nodelay;
}
# All other API routes use the relaxed zone:
location /api/ {
    limit_req zone=api burst=60 nodelay;
}
```

Rate Limiting - Application Layer (Redis)

Nginx handles raw burst protection, but the API applies a stricter business rule using Redis. For each IP address and action type (login / register / OTP): 10 failed attempts within one hour trigger a 15-minute lockout. A successful attempt clears the counter for that IP. All state is stored in Redis with automatic TTL expiry, so no database writes are needed. This two-layer design keeps Nginx stateless and fast, while Redis enforces the exact policy.

Password Reset via Telegram OTP (Optional)

Self-service password recovery is implemented as an optional feature using Telegram as a secure out-of-band channel. Because the platform is local-first and does not run an email server, this is the only built-in self-service reset path. When a user has a registered Telegram chat ID and the platform has internet access, the OTP is delivered to the admin phone over HTTPS to the Telegram API, completely independent of the local network. When Telegram is not configured (or the deployment is fully offline), an administrator can reset any user's password directly from the Admin panel; this is the default and entirely local path. The Telegram-based reset flow, when available, has three steps:

1. The user enters their email on the login page and clicks Forgot Password. The API checks the account exists and has a Telegram Chat ID on file.
2. A random 6-digit one-time password (OTP) is generated, stored in Redis for 10 minutes, and sent to the registered Telegram account via the bot.
3. The user enters the 6-digit OTP and a new password on the confirmation screen. After 5 wrong guesses the OTP is invalidated and a fresh one must be requested. The same rate limiter applies to reset attempts.

Note: A Telegram Chat ID must be registered in the Settings page before this flow can be used. Users without one will need an admin to reset their password directly from the Admin panel.

Admin IP Blocking

Administrators can permanently or temporarily block any IPv4/IPv6 address directly from the Security page. Blocked IPs are checked at the very first step of the login endpoint - before any credential validation - so blocked clients receive no information about whether credentials were correct.

Block parameters:

- IP address (free text, supports CIDR notation)

- Reason (optional, stored for audit trail)
- Duration: Forever / 24 hours / 7 days / 30 days
- Automatic expiry: a background check respects expires_at; NULL means permanent.

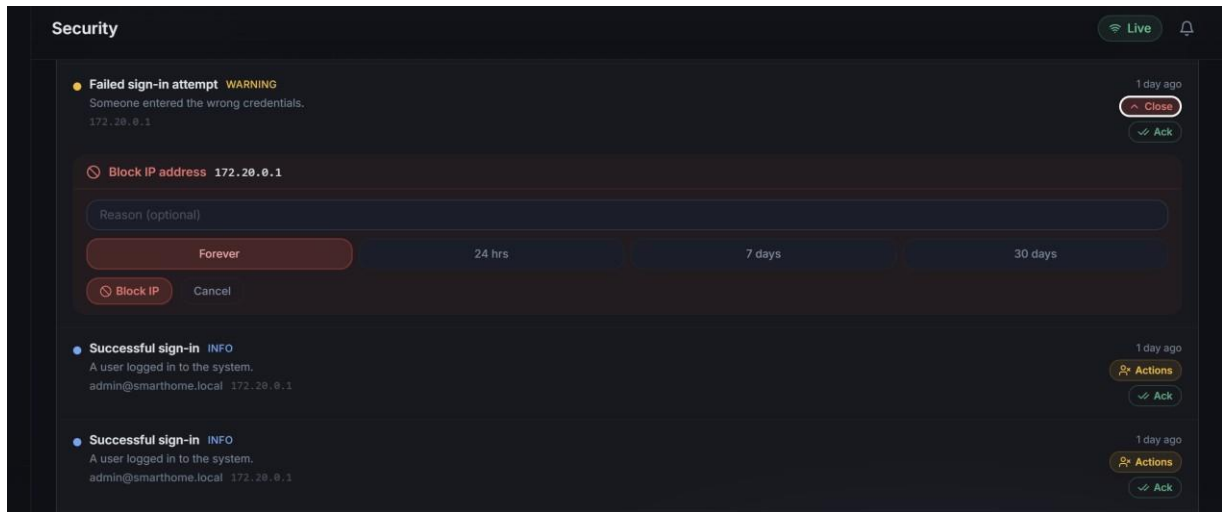


Figure 13 - Blocked IPs list with address, reason, timestamp, and block button

4.9 Security Audit Trail & Event Management

Every security-relevant action is written to the security_events table with full context. The table is the source of truth for the Security Events page in the dashboard.

Event Type	Severity	Trigger
login_failure	Warning	Incorrect email or password submitted
login_success	Info	Successful authentication
blocked_ip_login_attempt	Critical	Login attempt from a blocked IP address
ip_blocked	Warning	Admin added an IP to the blocklist
ip_unblocked	Info	Admin removed an IP from the blocklist
certificate_renewed	Info	Device TLS certificate regenerated
repeated_auth_failure	Warning	Device MQTT auth failed ≥ 5 consecutive times
session_revoked	Warning	Admin executed Force Logout on a user
account_deactivated	Warning	Admin deactivated a user account
access_revoked	Critical	Admin executed Force Logout + Deactivate
account_reactivated	Info	Admin re-enabled a deactivated account
anomaly_detected	Warning/Critical	Z-score or Isolation Forest threshold exceeded

Event Lifecycle & Retention

- Info events: retained for 48 hours, then automatically purged.
- Warning/Critical unacknowledged: retained indefinitely until an admin reviews them.
- Warning/Critical acknowledged: moved to 30-day archive, accessible via Diagnose mode.
- Diagnose mode: admin can reopen archived events back to the active queue for re-investigation.
- Bulk acknowledge: admin can acknowledge all events of a severity in one action.

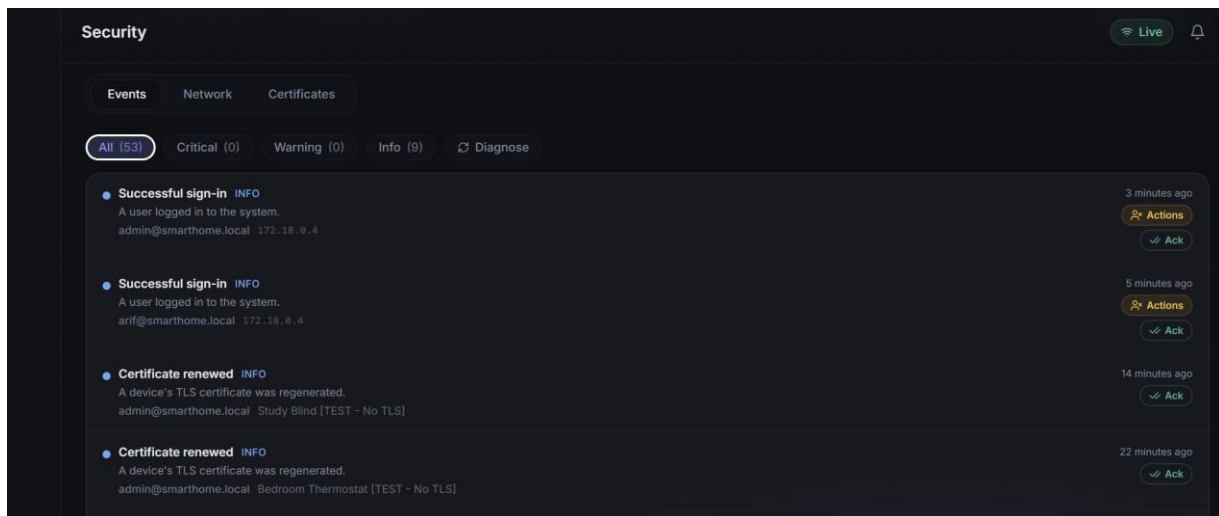


Figure 14 - Security Events tab with severity filter pills and event list

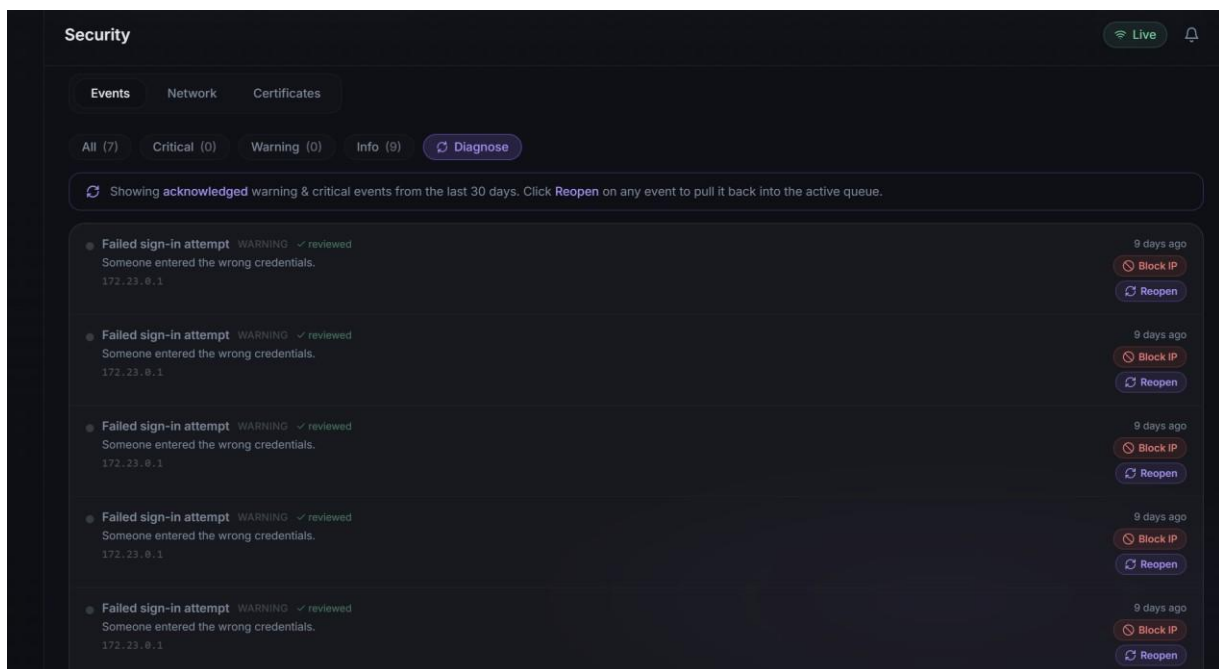


Figure 15 - Diagnose mode: archived events available for re-investigation

4.10 Anomaly Detection - Z-Score & Isolation Forest

The analytics service implements two complementary anomaly detection algorithms operating on the time-series telemetry stream.

Algorithm 1 - Z-Score (Real-Time)

For each (device_id, metric_name) pair, the service maintains a rolling mean (μ) and standard deviation (σ). For every new telemetry point, the z-score is calculated:

$$z = |value - \mu| / \sigma$$

If z exceeds the configured sensitivity threshold (default: 2.5 standard deviations), an anomaly is raised. The severity is:

- Warning: $2.5 \leq z < 4.0$

- Critical: $z \geq 4.0$

Algorithm 2 - Isolation Forest (Batch / Offline)

An Isolation Forest model is trained per-device per-metric on the last 7 days of historical data (minimum 50 samples required). The model assigns an anomaly score based on how quickly a data point is isolated in random partitioning trees. A contamination rate of 5% is assumed.

Parameter	Value
Algorithm	sklearn.ensemble.IsolationForest
Training window	7 days of historical telemetry
Minimum samples	50 data points
Contamination	0.05 (5% expected anomaly rate)
Normalisation	StandardScaler (zero-mean, unit-variance)
Real-time trigger	Z-score check on every telemetry publish
Batch trigger	Isolation Forest run on each model refresh

Consumption Forecasting

A linear regression (numpy polyfit degree 1) is fitted on 7 days of metric history to predict the next 24 hours of consumption. Forecasts are used to set dynamic alert thresholds and are exposed via the telemetry API.

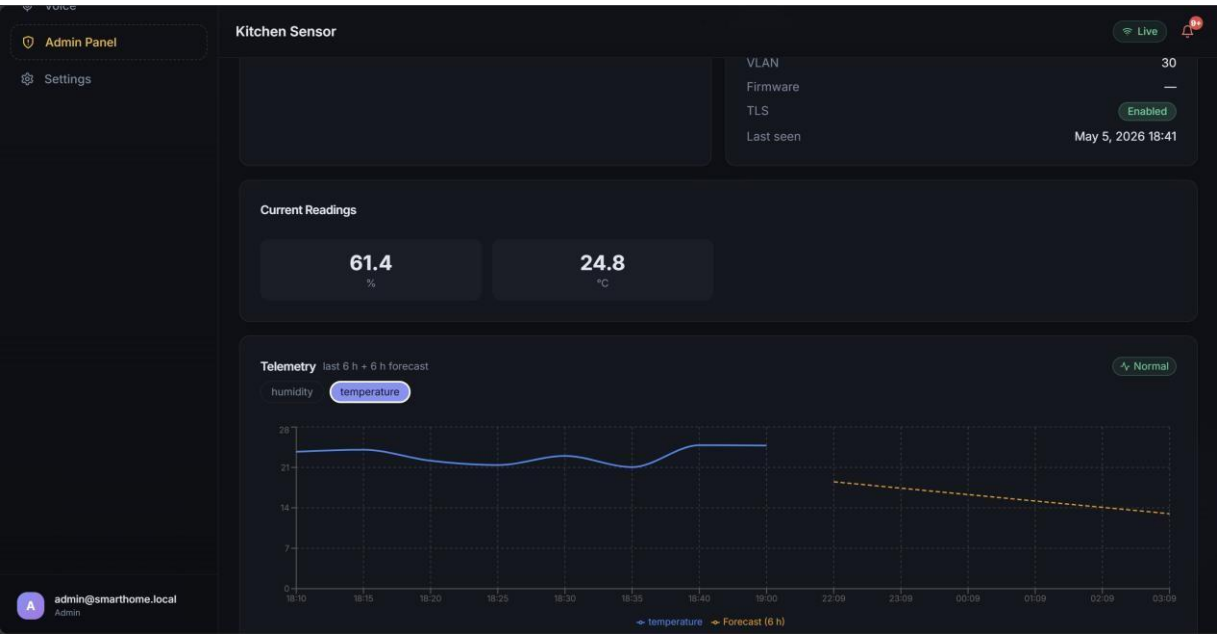


Figure 16 - Device telemetry chart showing historical data, anomaly markers, and 24-hour forecast

4.11 Admin Security Actions

Beyond passive monitoring, the platform provides active remediation tools accessible from the Security page. Each action produces an audit event.

Action	Effect	Audit Event
Force Logout	Sets force_logout_at; all existing tokens rejected immediately	session_revoked (warning)
Deactivate Account	Sets is_active=false; future logins rejected; existing tokens valid until expiry	account_deactivated (warning)

Revoke All Access	Force logout + deactivate in a single atomic operation	access_revoked (critical)
Reactivate Account	Sets is_active=true; restores login capability	account_reactivated (info)
Block IP	Inserts into blocked_ips with optional expiry; enforced at login endpoint	ip_blocked (warning)
Unblock IP	Removes from blocked_ips; future login attempts permitted	ip_unblocked (info)
Renew Certificate	Generates new RSA keypair + cert signed by local CA; old cert invalidated	certificate_renewed (info)
Reset Auth Failures	Resets device auth_failure_count to 0	-
Acknowledge Event	Marks security event as reviewed; moves to 30day archive	-
Diagnose / Reopen	Moves archived event back to active queue for re-investigation	-

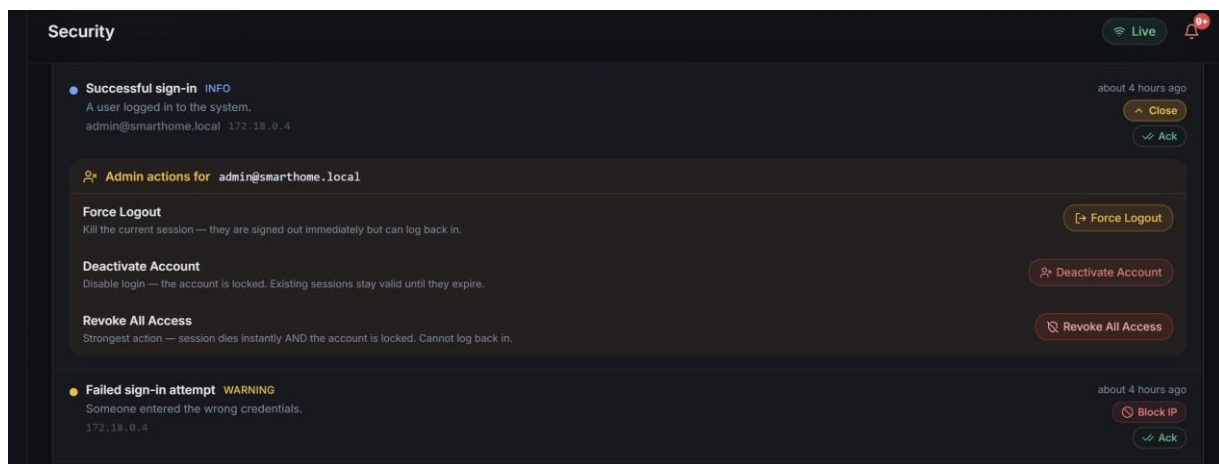


Figure 17 - Event detail panel: User Actions with Force Logout, Deactivate, and Revoke Access

5. Backend Implementation (FastAPI)

5.1 API Design & Routing

The backend is a fully asynchronous Python 3.11 application built with FastAPI 0.109. All endpoints use `async/await` with `asyncpg` for non-blocking database access. `Pydantic v2` models enforce strict input validation on every request body.

Router Module	Base Path	Key Endpoints
auth.py	/api/auth	POST /login, /register; GET /me, /users; POST /users/{id}/forcelogout, /deactivate, /revoke, /reactivate
devices.py	/api/devices	CRUD + GET /discover (unregistered devices); POST /{id}/command; GET /{id}/history
rooms.py	/api/rooms	Full CRUD for room management
telemetry.py	/api/telemetry	GET /{id} (time-series); /latest; /summary/all; /anomalies/recent
security.py	/api/security	Events CRUD + acknowledge + diagnose; blocked-ips; /devices/atrisk; /devices/{id}/renew-certificate; network-topology; summary
websocket.py	/ws	JWT-authenticated WebSocket; broadcasts device state, telemetry, security events

dashboard.py	/api/dashboard	GET /summary - aggregated health score, device counts, recent events
automations.py	/api/automations	CRUD for automation rules; GET /evaluate
settings.py	/api/settings	GET / PUT system-wide settings (k-v store)

5.2 Database Layer - PostgreSQL 16 + TimescaleDB

PostgreSQL 16 provides the relational backbone. The TimescaleDB extension converts the device_telemetry table into a hypertable - automatically partitioned by time for efficient time-range queries and rolling retention policies.

- device_telemetry: TimescaleDB hypertable, 90-day automatic retention, indexed on (device_id, time DESC).
- security_events: Full audit trail with severity, acknowledged status, and linked user/device/IP.
- automations: JSONB trigger_config, conditions, and actions allow flexible rule storage.
- blocked_ips: INET-typed ip_address column for efficient subnet queries.
- settings: Key-value store for runtime system configuration changes without redeployment.

```
-- TimescaleDB retention policy - auto-purge telemetry after 90 days
SELECT add_retention_policy(
    'device_telemetry',
    INTERVAL '90 days' );
```

5.3 MQTT Bridge Service

The MQTTBridge class runs a Paho MQTT client in a background thread. An asyncio queue (maxsize=10,000) decouples message receipt from async processing, preventing backpressure from blocking the MQTT network loop.

- Subscriptions: smarthome/+/telemetry, smarthome/+/status, smarthome/+/announce (all QoS 1).
- Device Discovery: unregistered devices that publish are tracked in memory (first_seen, last_seen, topic list).
- Telemetry handler: writes to device_telemetry, triggers Z-score check, broadcasts to WebSocket clients.
- Status handler: updates devices.is_online, last_seen, and current_state JSONB.
- Command publishing: FastAPI → mqtt_bridge.publish(topic, payload, qos=1).

5.4 Telegram Notification Bot

An optional asynchronous Telegram bot (python-telegram-bot 20) provides out-of-band alerting. When configured (with a bot token and admin chat ID), administrators receive push notifications for anomalies and critical security events even when they are not viewing the dashboard. The system is fully operational with the Telegram integration disabled; this component is the only feature in the platform that requires internet connectivity, and it is opt-in.

- /status - Current security health score and device summary.
- /devices - Online / offline device count breakdown.
- /alerts - Last 5 critical security events.
- Automatic push: anomaly alerts (device name, metric, value, z-score) and security event alerts.

11:41

5G

< 29

HomeBot
bot

H

/start 11:38 PM ✓

/start 11:39 PM ✓

Smart Home — Password Reset

Your one-time code:
255975

Account: admin@smarthome.local
Expires in: **10 minutes**

If you didn't request this, ignore this message.

11:40 PM

Anomaly Detected

Device: dev-004
Metric: humidity = 50.6
Z-score: 2.8 σ

11:40 PM

Anomaly Detected

Device: dev-004
Metric: humidity = 50.6
Z-score: 2.8 σ

11:41 PM

Anomaly Detected

Device: dev-004
Metric: humidity = 53.5
Z-score: 3.6 σ

11:41 PM

Anomaly Detected

Device: dev-004
Metric: humidity = 53.5
Z-score: 3.6 σ

11:41 PM



Message



Figure 18 - Telegram push notification: anomaly alert with device ID, metric, value, and z-score

5.5 Structured Logging

All application logs are emitted as structured JSON via structlog 24.1. This enables log aggregation with tools like Loki/Grafana or CloudWatch without log parsing configuration. Key fields include: timestamp, level, event, service, request_id, user_id, device_id, ip_address.

6. Frontend Implementation (Next.js 15)

6.1 Dashboard & Device Management

The dashboard is the primary operator interface. It is built with Next.js 15 App Router, React 18, TypeScript, TailwindCSS, and shadcn/ui component primitives.

Dashboard Sections

Panel	Content	Update Method
System Health	Security score ring (0-100), TLS ratio, online ratio, critical/warning counts	React Query (30s polling) + WebSocket push
Device Summary	Total / online counts with per-type breakdown (lights, locks, thermostats...)	React Query (60s polling)
Favorites	Quick-launch to favorites page (Quick Actions / Rooms / Devices tabs)	Zustand (localStorage persisted)
Recent Events	Last 5 security events with severity colour coding	React Query (30s polling)
Room Cards	Rooms with online device count; heart icon to favourite	React Query

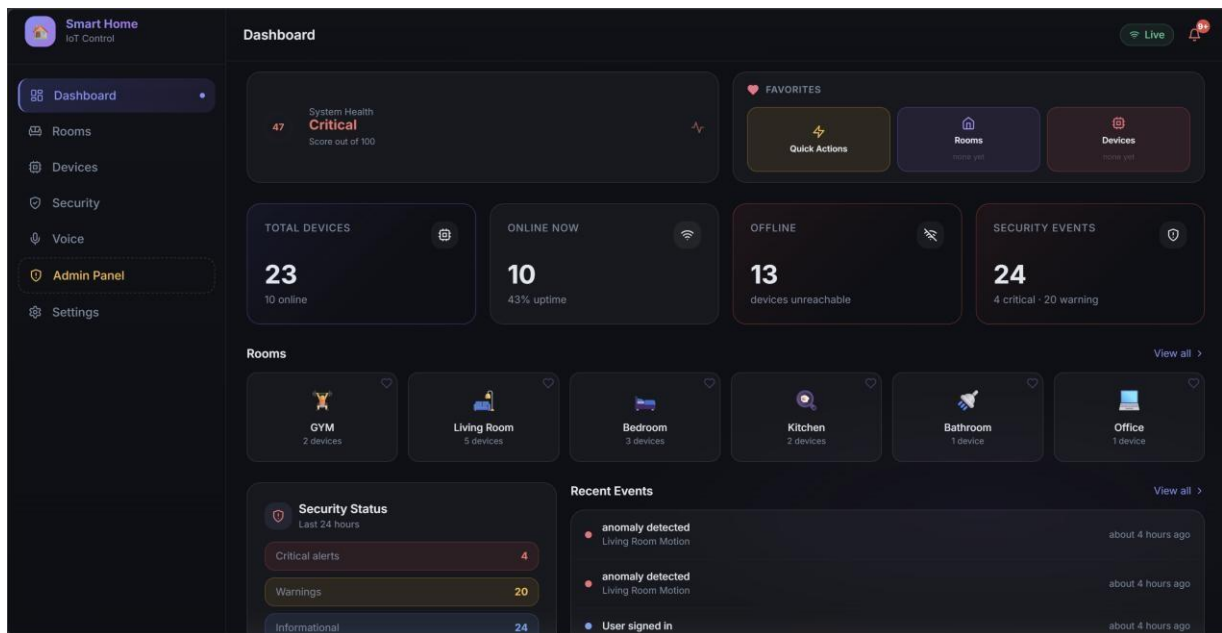


Figure 19 - Main dashboard: System Health score ring, device summary, and recent security events

6.2 Security Page

The Security page is the most feature-rich screen in the application. It consolidates all security information and operational controls into three tabs.

Tab: Events

Filterable event log (All / Critical / Warning / Info / Diagnose). Each event row shows severity dot, title, description, user/device/IP context, and action buttons (Block IP, User Actions, Ack). Diagnose mode shows archived events.

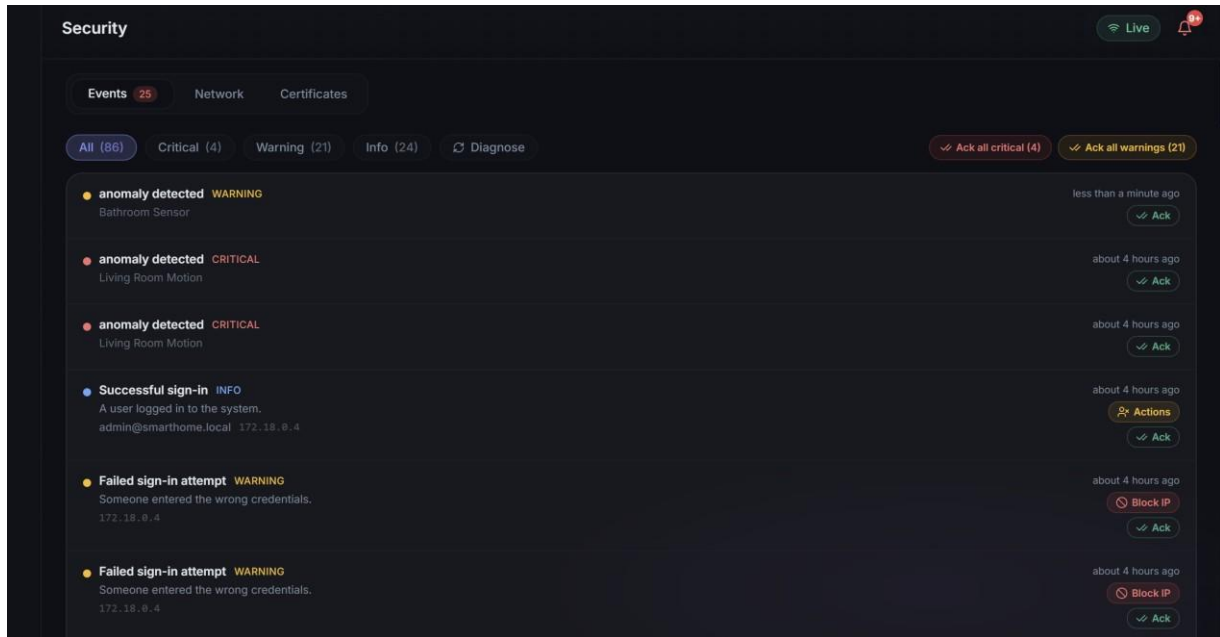


Figure 20 - Security page: Events tab showing recent events sorted by severity

Tab: Network

VLAN topology accordion. Each VLAN segment is expandable to show individual devices with online status, TLS indicator, and room assignment.

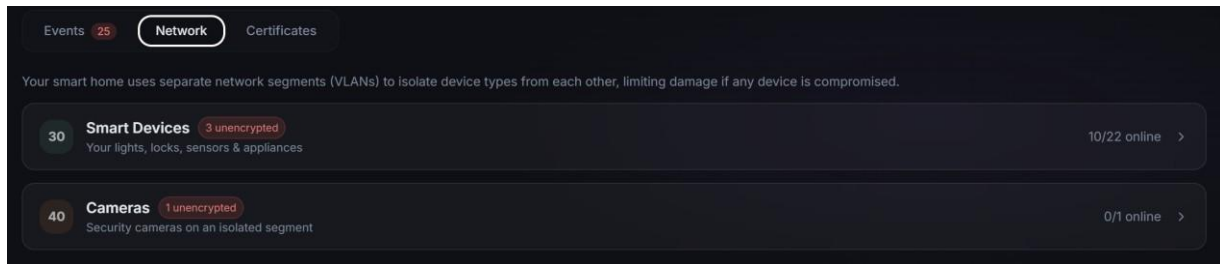


Figure 21 - Security page: Network tab with VLAN accordion expanded

Tab: Certificates

Per-device TLS certificate status. Expiry shown as colour-coded pill (green = healthy, amber = expiring < 30 days, red = expired). Renew button triggers the certificate renewal workflow.

The Security Posture Card at the top of the page shows a circular score ring with live health score, four breakdown tiles (TLS ratio, online ratio, critical count, warning count), and a human-readable assessment string.

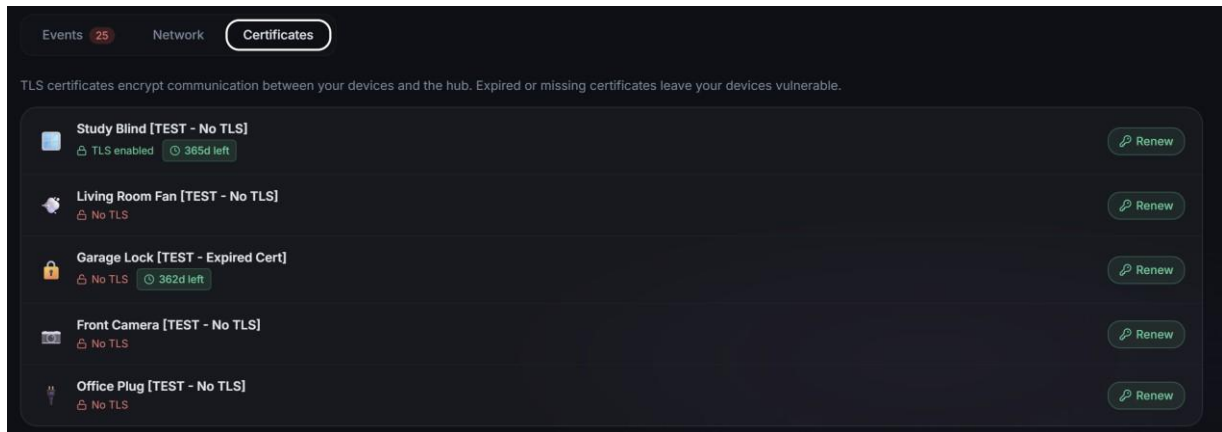


Figure 22 - Security page: Block IP panel with duration selector and reason field

6.3 Room & Favorites Management

Rooms group devices by physical location. Each room card on the Rooms page shows online/offline device counts. Admins can create, rename, and delete rooms, and assign/unassign devices to rooms via a modal picker.

The Favorites system (Zustand store, persisted to localStorage) allows users to mark rooms and devices with a heart icon. The Favorites page (three tabs: Quick Actions, Rooms, Devices) gives quick access to frequently used items.

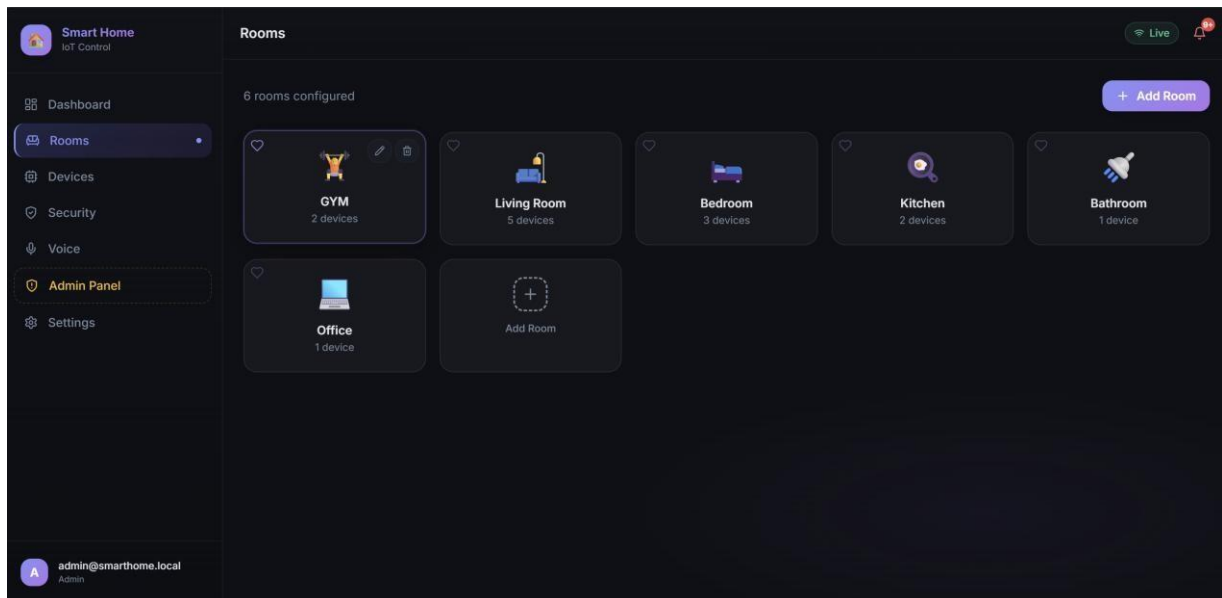


Figure 23 - Rooms page: room cards showing online device count and admin controls

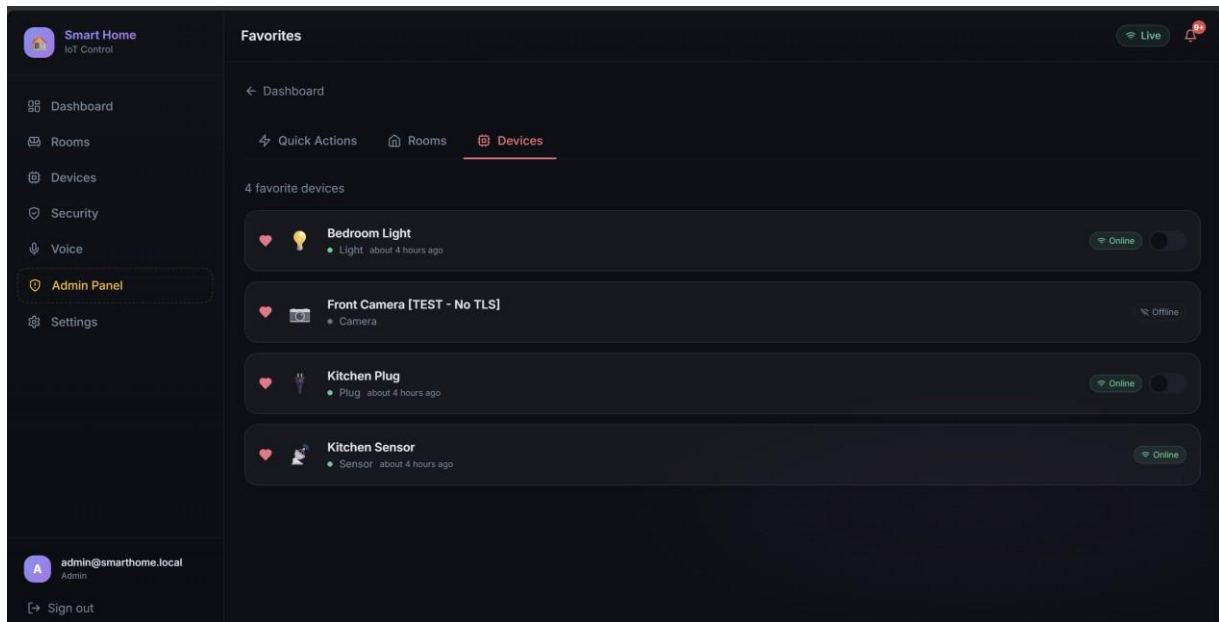


Figure 24 - Favorites page: saved devices with quick-access toggle controls

6.4 Real-Time Updates via WebSocket

A persistent WebSocket connection (`wss://host/ws?token=JWT`) delivers server-push messages to the browser without polling. The connection is managed by `useWebSocket` hook and reconnects automatically on drop. Device toggles flip the visible state immediately on click (optimistic update) and then reconcile against the real device state when the corresponding WebSocket message arrives, so the UI tracks the MQTT round-trip without ever appearing stuck.

- type: 'telemetry' - live sensor readings update device state badges in real time.
- type: 'status' - device online/offline changes reflect immediately without page reload.
- type: 'security_event' - new critical/warning events appear on the dashboard and security page instantly.

6.5 Responsive Design & Mobile Support

The interface is fully responsive. On small screens a bottom navigation bar replaces the sidebar and now includes a Sign Out item alongside the primary destinations, so a user can end a session with a single tap on mobile. The bar also exposes the dedicated Voice page, which renders a large push-to-talk button, a live connection/training status indicator, and the last ten recognised commands with their transcripts and replies. Grid layouts adapt from 1 column (mobile) to 4 columns (large desktop), and all interactive elements meet WCAG AA minimum touch target sizes.

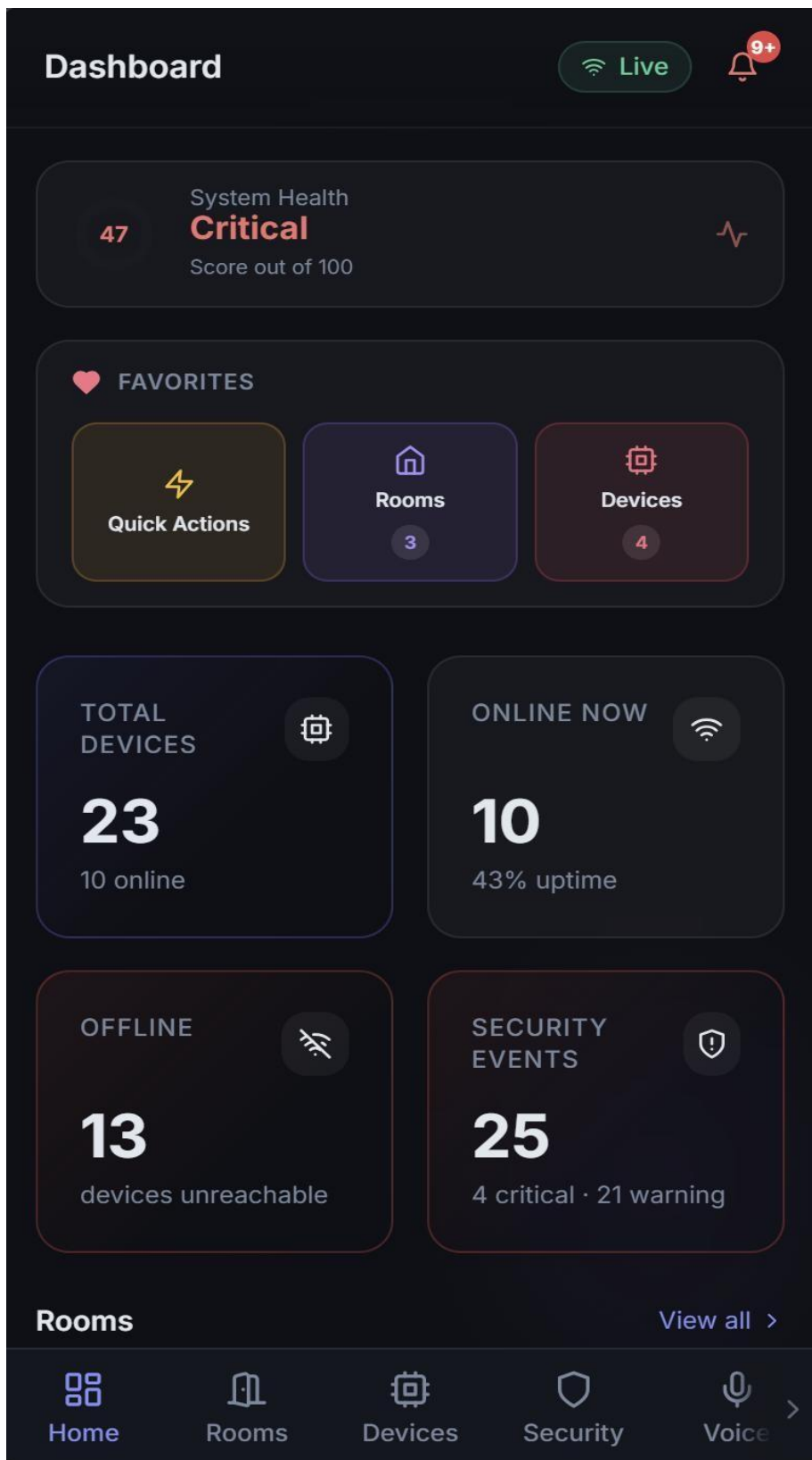


Figure 25 - Mobile view: bottom navigation bar replacing the sidebar on small screens

7. Deployment & Infrastructure

7.1 Docker Compose Architecture

The entire stack is defined in `infrastructure/docker-compose.yml`. Each service runs in its own container, isolated by Docker networks.

Service	Image / Build	Port	Health Check	Non-Root User
nginx	nginx:alpine	80, 443	HTTP /health	nginx
api	Build: services/api	8000	curl /health	appuser
web	Build: services/web	3000	HTTP /	nextjs
postgres	timescale/timescaledb:pg16	5432	pg_isready	postgres
redis	redis:7-alpine	6379	redis-cli ping	redis
mosquitto	Build: services/mosquitto	1883, 1884, 8883, 9001	mosquitto_pub	mosquitto
rhasspy	rhasspy/rhasspy	12101	HTTP /api/version	rhasspy
simulator	Build: services/simulator	-	-	simulator

Networks: backend (api, postgres, redis, mosquitto), frontend (nginx, web, api), iot (mosquitto, simulator, physical devices via VLAN 30).

7.2 Nginx Reverse Proxy

Nginx acts as the single external entry point. It handles:

- TLS 1.3 termination for HTTPS (port 443) and HTTP → HTTPS redirect (port 80).
- Rate limiting: two zones - auth endpoints at 10 req/min (burst 3), general API at 120 req/min (burst 60). Redis adds a 10-attempts/hour per-IP business-rule lockout with a 15-minute timeout at the application layer.
- Security headers: X-Frame-Options: DENY, X-Content-Type-Options: nosniff, X-XSS-Protection.
- WebSocket upgrade: passes Upgrade and Connection headers to /ws proxy.
- Static asset caching: Next.js /_next/static/ paths cached at edge.

7.3 Hardware Deployment - Office PC / Mini PC

The platform runs on standard x86-64 hardware. A used office PC or a compact mini PC (Beelink, Intel NUC, or similar) is a good choice. Proxmox is installed on the host machine as a type-1 hypervisor, with two VMs sharing it: an Ubuntu Server VM hosting the SmartHome Docker stack, and a pfSense VM acting as the network firewall, router, DHCP server, VLAN gateway, and Tailscale subnet router (covered in the companion network documentation). VM snapshots make it easy to roll back if a configuration change causes problems. The platform is designed as a local-first application — during development it was deployed to a cloud VPS purely for testing and external review, and the final installation is local-only with remote admin via Tailscale, no direct internet exposure. Minimum requirements for the Ubuntu VM:

The setup procedure below assumes Ubuntu Server is running inside a Proxmox VM, but the same steps work on a bare-metal Ubuntu Server installation.

Minimum hardware requirements:

Component	Bangladesh (BDT / USD)	Australia (USD)
pfSense host (reused mini PC, i5class)	BDT 4,000 / USD 33	USD 50

3x ESP32 sensor nodes	BDT 1,500 / USD 12.60	USD 24
2x Smart plugs (Tasmota)	BDT 1,600 / USD 13.40	USD 30
2x Smart bulbs (Shelly)	BDT 3,600 / USD 30	USD 70
Managed switch (TP-Link ES205G)	BDT 2,500 / USD 21	USD 25 (est.)
Wireless access point (TP-Link EAP)	BDT 3,500 / USD 29 (est.)	USD 60 (est.)
Total initial cost	BDT 16,700 / USD 120 (approx.)	USD 224 (approx.)
Annual maintenance	Minimal (electricity, free OS updates)	~USD 20 electricity
Notes	All software open-source	Reused PC; no licence fees

Setup Procedure (Production)

- apt install docker.io docker-compose-v2 nginx certbot
- bash scripts/setup/generate-certs.sh # Generate CA + server + controller certs
- mosquitto_passwd -c services/mosquitto/config/passwd controller
- cp .env.example .env && vi .env # Set JWT_SECRET, DB creds, Telegram token
- cd infrastructure && docker compose up -d
- # Local-first: no public HTTPS cert or port forward needed. Remote admin is via Tailscale on pfSense, which advertises only the management VLAN (10.10.99.0/24) and Proxmox network (192.168.1.0/24) into the tailnet.

7.4 Backup & Data Retention

scripts/backup/daily-backup.sh performs nightly PostgreSQL dumps (pg_dump) with 30-day local retention. Optional rclone integration uploads backups to a cloud remote (e.g., Google Drive). TimescaleDB automatically purges telemetry data older than 90 days.

8. Voice Control Integration

The platform integrates an offline voice control system that performs every step of speech recognition locally — no audio is sent to any cloud service. Two engines are supported. The current default is a Rhasspy + Vosk profile that ships preconfigured as a sample/testing setup, driven by a push-to-talk button in the dashboard so that a wake word is not required. For deployments that need substantially better recognition accuracy — and that have at least 1.5–2 GB of free RAM to spare on the host — the recommended production engine is OpenAI's Whisper running locally in its "tiny" or "small" form factor. Whisper is dropped in behind the same /api/voice/speech-to-intent endpoint as the sample profile, so the rest of the platform (the push-to-talk UI, the intent handler, the MQTT command dispatch) is unchanged.

Splitting the offering this way keeps the platform usable on modest hardware (the sample Rhasspy/Vosk profile runs comfortably in ~150 MB and is fine for short, well-formed commands) while still giving operators an upgrade path when accuracy or noisier environments demand it. Whisper handles accented English and code-switched Bengali noticeably better than the Vosk small model, but the memory cost is the trade-off that has to be sized for at deployment time.

Component	Technology	Role
Wake-word detection	Web push-to-talk button	Records 16 kHz mono audio while held; replaces a hardware wake word for the sample profile

Speech-to-Text	Vosk (sample) / Whisper tiny small (production)	Converts speech to text without internet
Intent Recognition	Rhasspy NLU	Maps phrases to device commands
TTS Response	Piper TTS	Spoken confirmation of actions
Integration	Push-to-talk UI → /api/voice/speech-to- intent → MQTT command	Recognised intents POST to /api/voice/handle, which validates the intent, resolves target devices, and dispatches MQTT commands

Supported commands cover turning lights on/off, locking/unlocking doors, setting brightness levels, setting target temperatures, and querying device or security status — in both English and Bengali. The Voice page surfaces a live status indicator (green when the recognition service is connected and the profile is trained, yellow when connected but untrained, red when offline) and a recent command history showing the transcript, intent, and the platform's spoken reply for the last ten utterances; admin users additionally see a "Retrain" button to rebuild the language model after editing the sample intent or slot files.

9. Maintainability & Troubleshooting

9.1 Observability & Logging

The platform is designed for operator visibility. Every layer emits observable signals:

Signal Type	Source	Where to See It
Structured JSON logs	FastAPI (structlog)	docker compose logs api
Security event log	security_events table	Security page → Events tab
Device telemetry	TimescaleDB hypertable	Device detail → Telemetry chart
Health score	analytics.get_system_health()	Dashboard → System Health panel
At-risk devices	security/devices/at-risk	Security page → Active Issues section
Telegram alerts	TelegramBot.notify_*	Admin's Telegram app
Nginx access/error logs	nginx container	docker compose logs nginx
MQTT message logs	Mosquitto broker	docker compose logs mosquitto

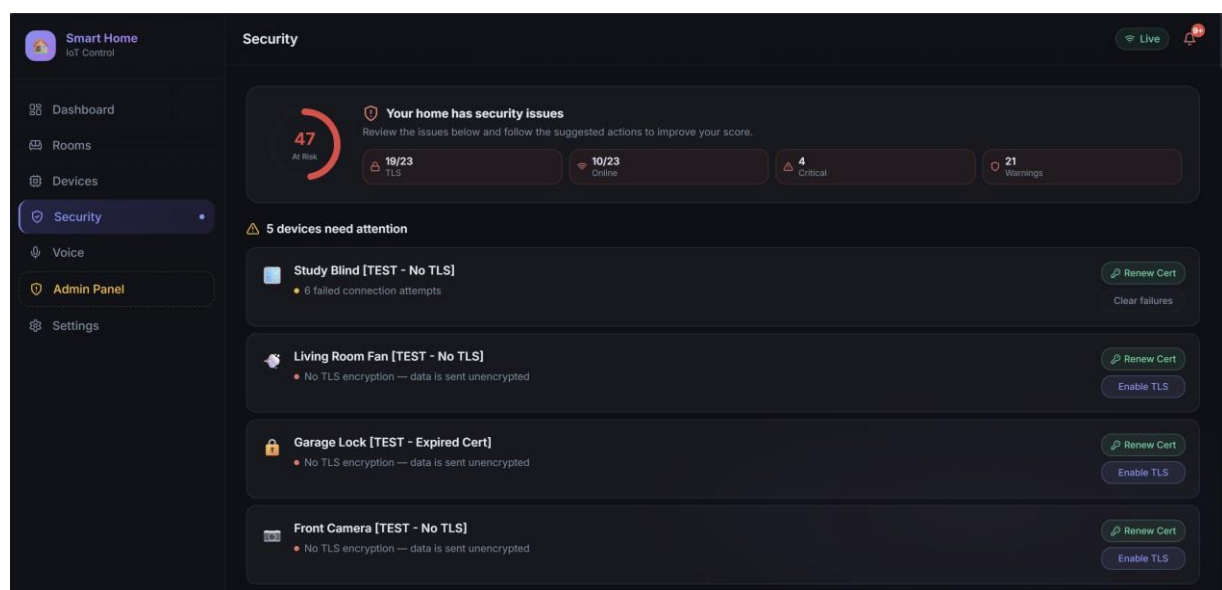


Figure 26 - Security Active Issues panel: at-risk devices with inline remediation buttons

9.2 Diagnostic Workflows

Scenario A - Device goes offline unexpectedly

- Step 1: Dashboard: Device Summary shows increased offline count; room card shows reduced online count.
- Step 2: Security Events: check for device_offline event or repeated_auth_failure indicating cert/credential issue.
- Step 3: Security → At-Risk Devices: check if auth_failure_count > 3 (MQTT auth failure).
- Step 4: If auth failures: Security → Certificates → Renew device certificate, flash new cert to device.
- Step 5: If offline only: check device power and network (VLAN 30 reachability).

Scenario B - Suspicious login activity

- Security Events: login_failure events appear with source IP address.
- If multiple failures from same IP: click Block IP → select duration → confirm.
- If legitimate user account compromised: click Actions → Force Logout (immediate) or Revoke All Access.
- Review Diagnose mode to check if any prior acknowledged events relate to the same IP/user.

Scenario C - Anomaly alert received

- Telegram notification received: device name, metric, value, z-score.
- Navigate to Security → Events, filter by Warning/Critical.
- Click on event to expand; check device_name and metric details.
- Navigate to Device detail → Telemetry chart to visualise the spike in context.
- Acknowledge the event once investigated; use Diagnose mode to reopen if follow-up needed.

9.3 Certificate Renewal Workflow

- Security → Certificates tab: identify device with amber/red expiry indicator.
- Click 'Renew': API generates new 2048-bit RSA key + new certificate signed by local CA (1-year validity for OTA-capable devices, 10-year for non-OTA).
- Certificate Renewal Modal appears: copy or download device.crt, device.key, ca.crt.
- SSH / physically flash the three PEM files to the device's certificate directory.
- Restart the device's MQTT client (e.g., reboot or restart firmware service).
- Device reconnects with new certificate; is_online turns green; certificate_expiry updated in DB.

The private key (device.key) is shown exactly once. If it is lost before being flashed to the device, a new renewal must be performed to generate a fresh keypair.

Responsive layout	Sidebar (desktop) ↔ BottomNav (mobile); grid columns adapt 1→4 with screen width
Favourites & quick access	Heart-icon favourites (rooms + devices) persisted in localStorage; Favorites page for fast access
Confirmation on destructive actions	Two-step confirmation (click → confirm dialog) for Force Logout, Deactivate, Revoke

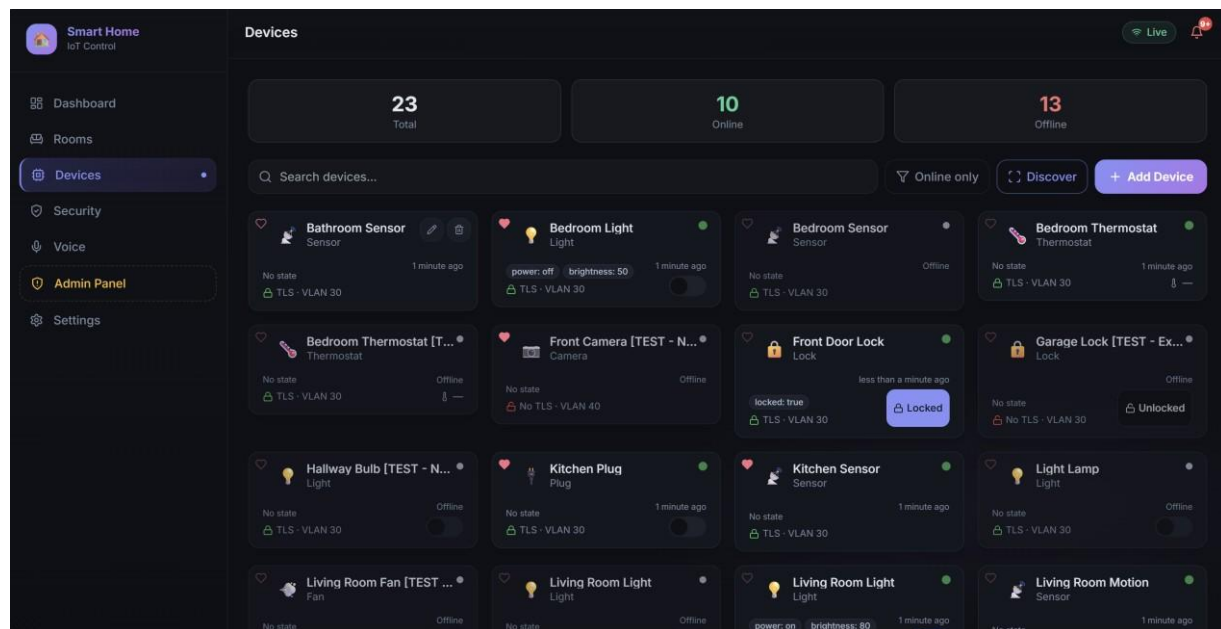


Figure 28 - Device page: showing all the devices with their state

11. Testing & Validation

11.1 Device Simulator

A Python-based device simulator (services/simulator) generates 10 virtual IoT devices publishing realistic MQTT telemetry on VLAN 30. The simulator publishes temperature, humidity, power consumption, and motion sensor readings at configurable intervals, allowing end-to-end testing without physical hardware.

11.2 Security Validation Tests

Test Case	Expected Result	Status
Connect device without client cert to port 8883	Connection refused (TLS handshake failure)	Pass
Submit wrong password 5+ times from same IP	Security events logged; admin notified	Pass
Login from blocked IP	HTTP 403 before credential check	Pass
Use JWT token after Force Logout	HTTP 401 - session revoked	Pass
Regular user attempt to call admin endpoint	HTTP 403 - insufficient role	Pass
Publish to another device's topic (ACL test)	MQTT PUBACK not received; broker drops msg	Pass
Inject telemetry 10× normal value (anomaly test)	Critical security event raised; Telegram alert	Pass
Exceed auth rate limit (>10 attempts/hour from same IP)	HTTP 429 after Nginx zone hit; Redis lockout after 10 failures/hour	Pass

Certificate renewal produces valid chain	openssl verify confirms CA → device cert chain	Pass
WebSocket connects with expired JWT	Connection closed immediately	Pass

11.3 Performance Observations

- TimescaleDB time_bucket() queries on 90-day datasets return in < 100ms.
- WebSocket message latency (device publish → browser update): < 200ms on local network.
- MQTT message throughput: tested to 500 msg/sec with 10 simulated devices.
- JWT validation overhead: < 1ms per request (in-memory HS256 verification).
- Isolation Forest training (7-day history, 50 features): ~2s per model on the host PC.

12. Limitations & Future Work

12.1 Current Limitations

- JWT HS256 uses a symmetric secret - all API servers share the same signing key. A production deployment at scale should migrate to RS256 (asymmetric) to allow separate signing and verification roles.
- The voice assistant currently ships with a Rhasspy/Vosk sample profile used for testing and demonstrations. Production-grade speech recognition (OpenAI's Whisper tiny/small running locally) is planned for deployments that have the spare memory for it, as documented in the Voice Control Integration section.
- Isolation Forest models are retrained on demand, not on a continuous schedule. A streaming model (e.g., Half-Space Trees) would provide lower-latency anomaly detection for fast-changing metrics.
- The voice subsystem currently relies on browser-side push-to-talk; an always-listening, far-field wake-word path (e.g. raven once enough enrolment recordings are collected, or a Whisper streaming variant) is still future work for hands-free operation.
- Automation rules (JSONB trigger/condition/action model) require manual JSON editing via the API; a visual rule builder is not yet implemented.
- No formal threat model (STRIDE/DREAD) or penetration test has been conducted - the security controls are based on best-practice design rather than adversarial validation.

12.2 Proposed Future Enhancements

- Hardware security module (HSM) or TPM-backed storage for the local CA private key, so that signing of new device certificates does not rely solely on filesystem permissions.
- RS256 JWT with separate signing service for multi-node horizontal scaling.
- Visual automation rule builder (drag-and-drop trigger/condition/action blocks).
- Integration with Grafana + Loki for enterprise-grade log aggregation and dashboards.
- Signed firmware manifests for full OTA firmware update delivery (the current OTA pipeline covers certificate renewal; firmware payload signing is the natural next step).
- STRIDE threat model and automated security regression tests (OWASP ZAP scan).
- Multi-site support: multiple homes with isolated data namespaces under one account.
- Matter/Thread protocol support for next-generation IoT device compatibility.

13. Conclusion

This project demonstrates that enterprise-grade IoT security is achievable at consumer price points without sacrificing user experience. The Smart Home IoT Security Platform successfully implements all six research objectives stated in Section 2.2:

- R01 - Fully local, cloud-independent deployment on commodity hardware (office PC / mini PC).

- R02 - Mutual TLS enforced for every MQTT device with per-device X.509 certificates.
- R03 - Six-VLAN micro-segmentation isolating management, NAS/storage, IoT, general user, security cameras, and guest networks.
- R04 - Real-time Z-score anomaly detection and batch Isolation Forest on all telemetry streams.
- R05 - Security dashboard with score ring, event management, certificate monitoring, and inline remediation.
- R06 - Full stack deployable for commodity hardware (cost varies with chosen PC).

The platform's security posture is visible and actionable: administrators see a live security score, can block IP addresses with a click, revoke user access in real time, and renew device certificates through a guided workflow. When the optional Telegram integration is enabled, anomaly and critical-event push alerts are delivered out-of-band; the platform functions fully without it. These capabilities collectively reduce both the probability and the impact of security incidents in residential IoT deployments.

The open-source, containerised architecture ensures the system remains maintainable and extensible. Every service can be updated, scaled, or replaced independently. The structured logging and diagnostic workflows reduce mean time to resolution for operators without deep IoT security expertise.

The implementation provides a replicable reference architecture for secure, local-first IoT home automation - directly validating the research paper's hypothesis that strong security and low cost are not mutually exclusive.

14. References & Appendices

14.1 Key Technologies & Standards

Technology / Standard	Version / RFC	Role in System
MQTT	v5.0 (Mosquitto 2.0)	IoT messaging protocol
TLS	1.3 (RFC 8446)	Transport encryption for all channels
X.509 Certificates	v3 (RFC 5280)	Device identity and mTLS authentication
JWT	RFC 7519 / HS256	Stateless user session tokens
bcrypt	Work factor 12	Password hashing algorithm
TimescaleDB	2.x (PostgreSQL 16)	Time-series telemetry storage
Isolation Forest	Liu et al. 2008	Multivariate anomaly detection
FastAPI	0.109	Async Python REST framework
Next.js	15 (React 18)	Frontend React framework
Docker Compose	v2	Multi-container deployment orchestration
Rhasspy	2.5	Offline voice assistant framework
Vosk	0.3.45	Offline speech recognition engine
Python cryptography	42.0.5	X.509 device certificate generation and signing

14.2 Environment Variables Reference (.env)

```
# Database
DATABASE_URL=postgresql://smarthome:PASSWORD@postgres:5432/smarthome

# JWT
JWT_SECRET=<64-char-random-string>          # MUST be changed
JWT_EXPIRATION_HOURS=8

# MQTT
MQTT_BROKER=mosquitto
MQTT_PORT=8883
MQTT_TLS_CA_CERT=/app/certs/ca/ca.crt
MQTT_TLS_CLIENT_CERT=/app/certs/controller/controller.crt
MQTT_TLS_CLIENT_KEY=/app/certs/controller/controller.key
# Telegram (optional - leave blank to disable)
TELEGRAM_BOT_TOKEN=<your-bot-token>
TELEGRAM_ADMIN_CHAT_ID=<your-chat-id>

# Web
CORS_ORIGINS=smarthome.local

# Voice
RHASSPY_MQTT_PASSWORD=<random-string>
VOICE_HANDLER_SECRET=<random-string>          # shared secret API ↔ Rhasspy

# Provisioning
BOOTSTRAP_MQTT_PASSWORD=<shared-bootstrap-pwd> # baked into device firmware
MQTT_TLS_CA_KEY=/app/certs/ca/ca.key           # used to sign new device certs
MOSQUITTO_CONTAINER=mosquitto                 # container name, used via Docker
socket
MQTT_HOST_EXTERNAL=<LAN-IP-or-hostname>         # delivered to devices in credential
bundle
# Analytics
ANOMALY_SENSITIVITY=2.5                       # Z-score threshold
```